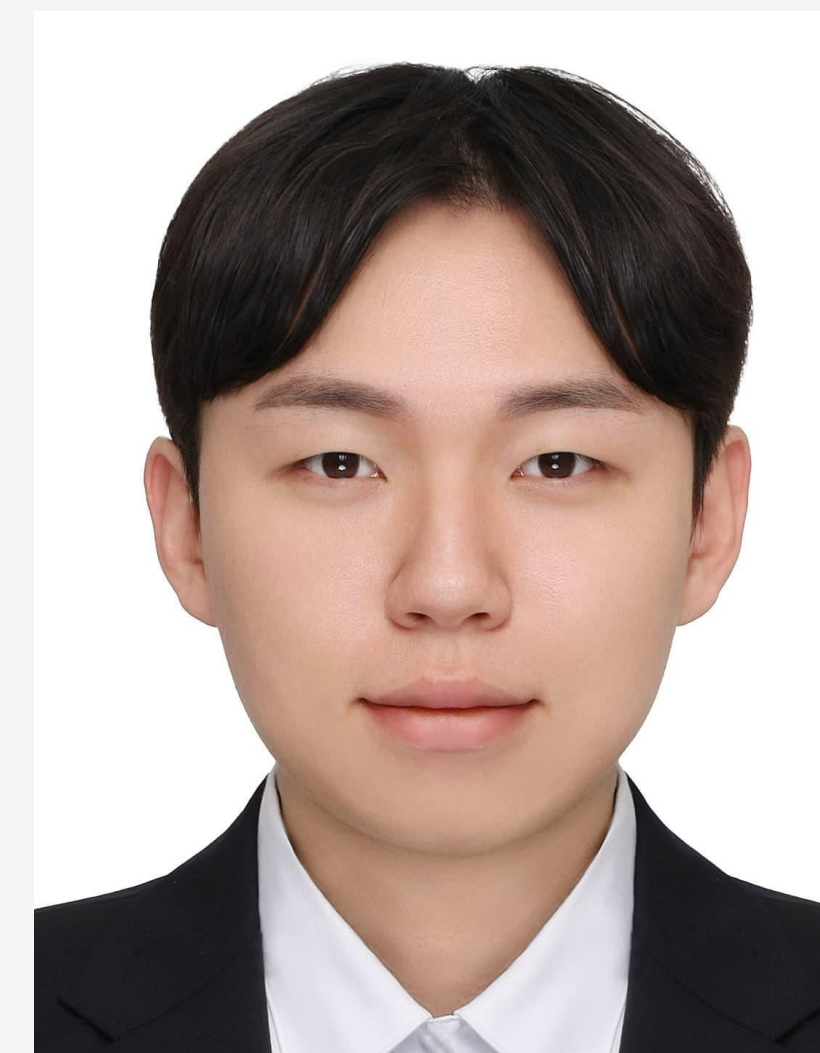


Portfolio

안녕하세요. 성장하는 개발자 홍형준입니다.

저는 옛지에서 클라우드까지, AI 모델 개발부터 배포·운영까지 전 과정을 직접 연결해본 엔지니어입니다. 단순히 코드를 작성하는 것을 넘어, 시스템이 실제로 동작하고 안정적으로 운영되기까지의 전체 흐름을 몸으로 익혔다는 것이 저만의 강점입니다.



Software Engineer.

홍형준

E-Mail.

hhj4498@naver.com

Contents

Chapter 1.	Creaply 게임과 크레이터를 연결하는 플랫폼
Chapter 2.	게임 현지화 번역 · 협업 검수 워크스페이스
Chapter 3.	더 플레이플러스 CS
Chapter 4.	Gen AI 기반 인터랙티브 AI 역사 교과서
Chapter 5.	CNN 기반 노인 낙상 감지 실시간 모니터링
Chapter 6.	Spring Boot 기반 AI 블로그 서비스

엣지 디바이스부터 웹까지, 서비스 전 구간을 구현



주력

언어 Python · Java
백엔드 Spring Boot · Django
클라우드·인프라 AWS (ECS Fargate · Bedrock · RDS · VPC) · Terraform · CI/CD
AI RAG 구축 (LangChain) · Bedrock Tool Calling · 모델 경량화 (Pruning · TFLite · INT8)

활용

백엔드 Spring Security·JWT · Spring Data JPA · Spring Cloud Gateway
DB PostgreSQL · MariaDB · Redis
실시간 WebSocket·STOMP · SSE
엣지·ML Raspberry Pi 4 · Coral TPU · PyTorch · OpenCV · YOLOv8

도구 · 협업

GitHub · Jenkins · GitHub Actions · Swagger · Notion · Figma · ERDcloud

PROJECT.1

Creaply

게임과 크레이터를 연결하는 플랫폼

01



프로젝트 기간	2026.06.18 ~
프로젝트 현황	개발 중
팀 구성	개인 프로젝트
역할	풀스택 개발
도메인	https://www.creaply.com/

게임사

캠페인을 만들고 결제하고, 지원자를 검수해 정산하는 스튜디오 워크플로우

1 가입 · 온보딩
게임사 이름 + 사업자등록번호 검증

2 캠페인 생성 · 결제
4단계 마법사 → 예산 전액 선결제

3 모집 · 검수
선착순 배정 → 제출 콘텐츠 승인/반려 요청

4 리포트 · 환불
성과 리포트 PDF, 미집행 예산 환불

섹션별 화면

ST-01 게임사 대시보드
ST-02 캠페인 탐색 (내 것 + 타사)
ST-03 캠페인 생성 마법사
ST-04·05 캠페인 결제 · 결제 결과
ST-06 내 캠페인 상세 (4탭 + 리포트)
ST-07 캠페인 수정

ST-08 타사 캠페인 상세
ST-09 지원 상세 · 채팅
ST-10 예산 관리 · 환불 신청
ST-11 게임사 프로필 · 계정
ST-12 관리자 문의

크리에이터

채널을 인증하고 지원해 콘텐츠를 제출하면, 검수 즉시 정산되는 크리에이터 워크플로우



섹션별 화면

- CR-01 오늘의 작업실 (대시보드)
- CR-02 캠페인 탐색 (참여 가능 필터)
- CR-03 캠페인 지원 · 선착순 배정
- CR-04 내 지원 · 제출 목록
- CR-05 지원 상세 · 콘텐츠 제출 · 성과
- CR-06 채널 관리 · 소유 인증

- CR-07 내 등급 분석
- CR-08 수익 · 계좌 · 출금 · 세무 신원
- CR-09 프로필 · 포트폴리오 · 계정
- CR-10·11 관리자 문의 · 반려 건 문의
- ADM 관리자 콘솔 12 뷰 (검수 · 정산 · 보너스 · 감사)

Why — 게임사 마케팅 실무에서 겪은 인플루언서 캠페인의 문제

구독자 수로 감을 잡고, 성과 확인 없이 선지급하던 인플루언서 마케팅

기존 인플루언서 마케팅은 이런 문제가 있었어요

감으로 매기는 견적

구독자 수만 보고 그때그때 협상하는 단가, 표준이 없어 캠페인마다 다시 흥정해야 했어요.

불투명한 정산

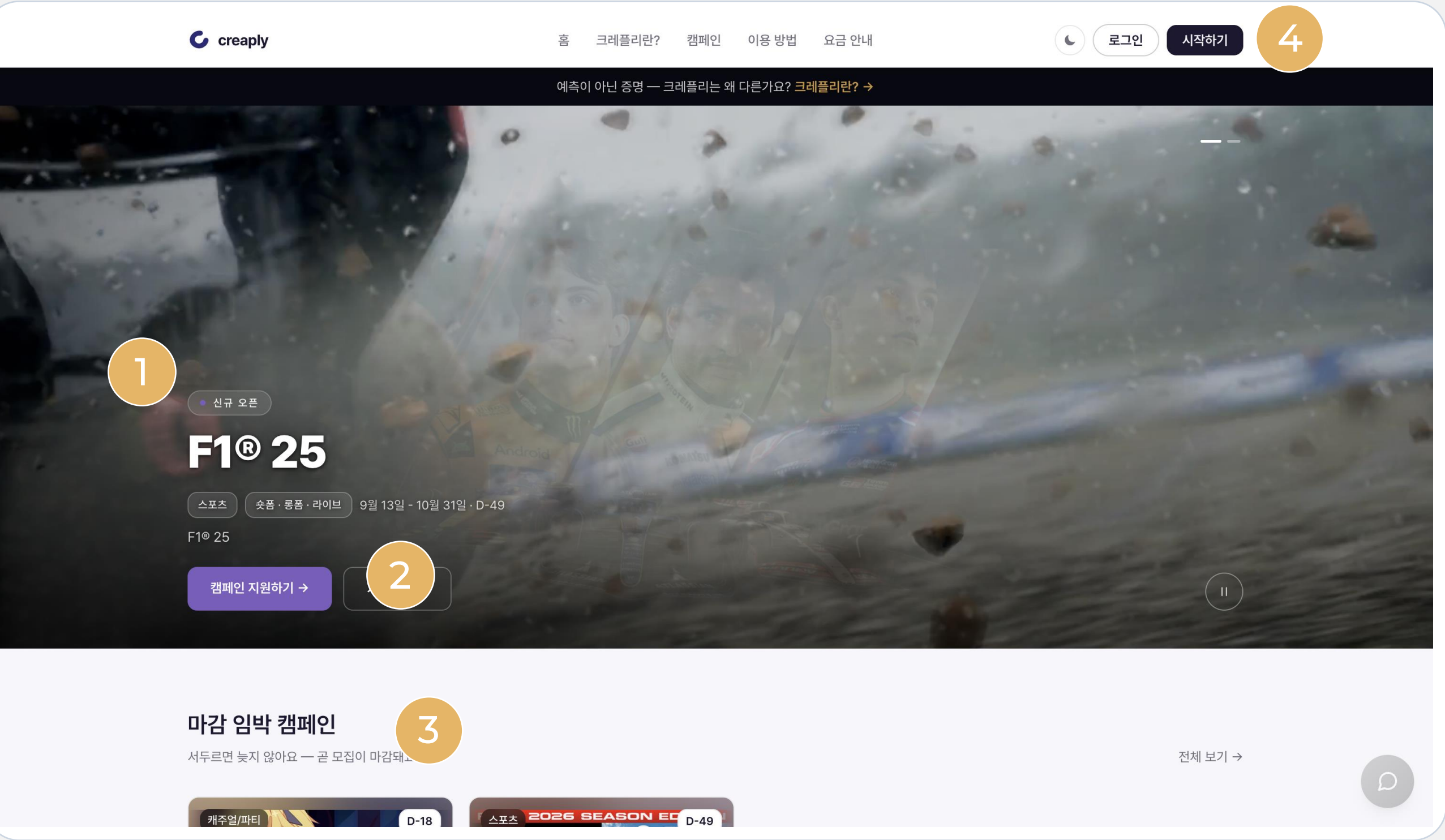
콘텐츠가 실제로 얼마나 성과를 냈는지 확인되지 않은 채 선지급되는 경우가 많았어요.

검증 안 된 크리에이터

채널을 실제로 소유하고 있는지, 조회수가 진짜인지 확인할 방법이 없었어요.

감으로 매기는 견적	불투명한 정산	검증 안 된 크리에이터
구독자 수만 보고 캠페인마다 단가 재흥정 표준이 없어 견적에 며칠, 근거 설명 불가	성과 확인 없이 선지급, 집행 내역은 엑셀 어디에 얼마가 쓰였는지 양쪽 다 모름	채널 소유 · 조회수 진위 확인 불가 성과 보고는 크리에이터 캡처에 의존
How + Result : 플랫폼 × 등급 × 콘텐츠 표준 단가표	How + Result : 선결제 에스스로 + 승인 시에만 정산	How + Result : OAuth 소유 인증 + 성과 자동 수집

기능 소개 01 — 공개 화면 · 랜딩 · 캠페인 둘러보기 · 캠페인 상세



1 모집 중 캠페인 히어로 캐러셀

모집 중 캠페인이 자동 순환, 게임 트레일러를 HLS로 재생
장르 · 콘텐츠 타입 · 모집 기간 · D-day 배지

2 역할별 CTA 분기

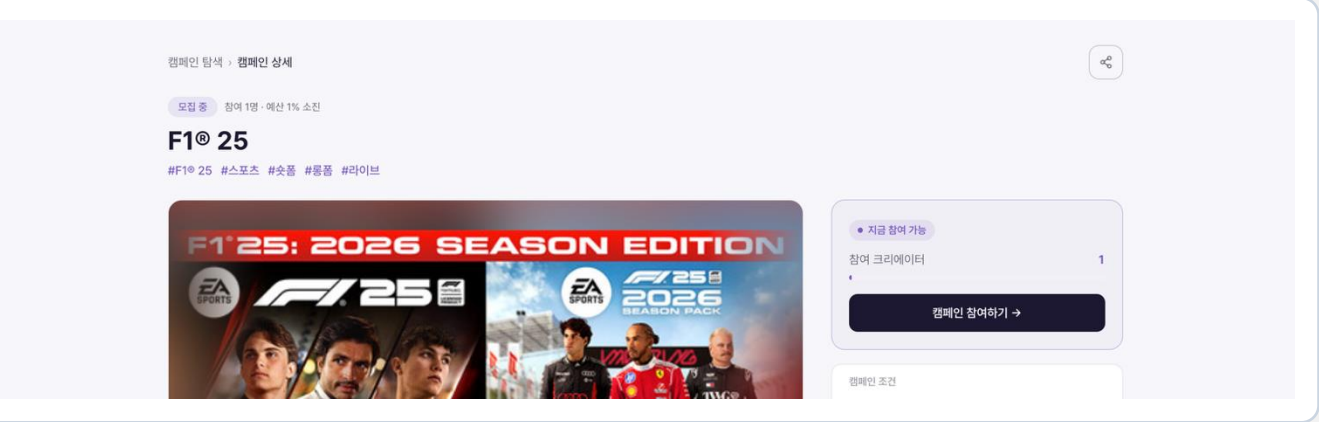
"캠페인 지원하기"는 로그인 상태와 역할에 따라
지원 화면 / 로그인 / 스튜디오로 리다이렉트

3 마감 임박 · 신규 오픈 캠페인

캠페인 둘러보기: 진행 중/마감 탭, 참여 많은 순 · 최신순 ·
마감 임박순 정렬, 검색, 예산 소진 바 (get_campaign_public_stats)

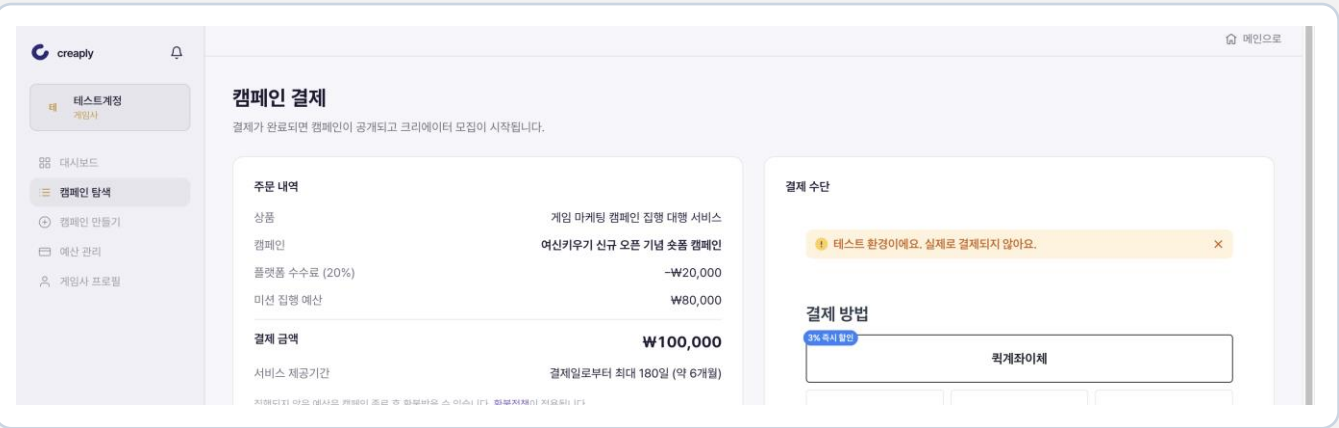
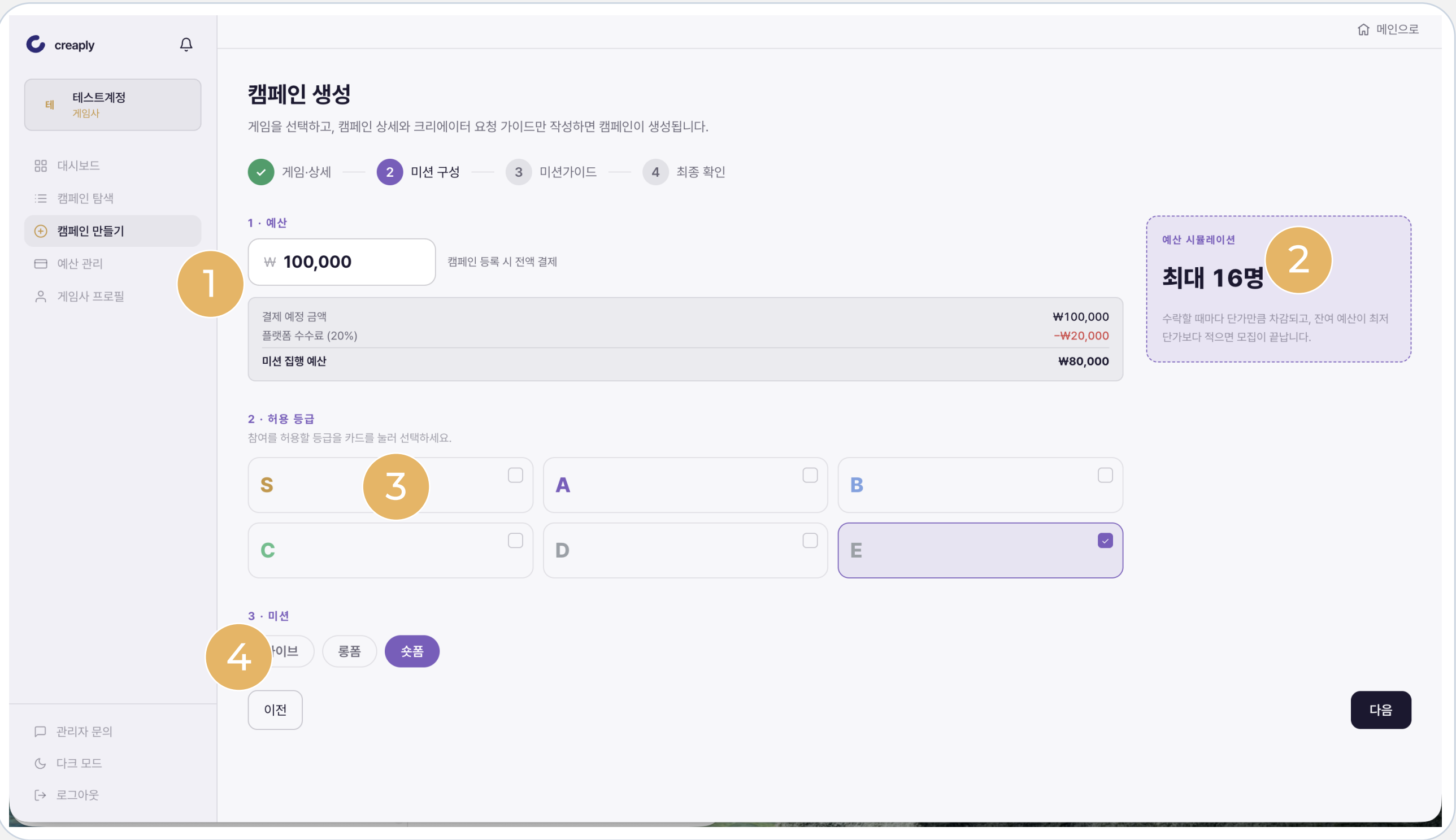
4 공통 셀

다크 모드 · 알림 · 문의 위젯, 약관 동의 게이트 · 온보딩
역할별 리다이렉트 규칙은 middleware 한 곳에서 처리



캠페인 상세 (/campaigns/{id})
미션 · 등급별 최대 정산액 · 참여 방법 · 트레일러
로그인 크리에이터에게는 "내 등급 참여 가능"
배지와 실수령 예정액이 추가로 보임

기능 소개 02 — 게임사 · 캠페인 생성 마법사 · 토스페이먼츠 결제



캠페인 결제 (/studio/campaigns/{id}/pay)
주문 내역 · 수수료 · 서비스 제공기간을 결제
직전에 다시 확인, 토스페이먼츠 결제위젯으로
카드 · 계좌이체 · 간편결제 · 가상계좌

1 예산 입력 → 실시간 분할 계산

결제 예정액에서 플랫폼 수수료 20% · 집행 예산을 즉시 계산
지금은 지급 풀 70% / 보너스 풀 10%까지 3분할로 표시

2 모집 인원 시뮬레이션

허용 등급의 단가로 최대 몇 명까지 모집 가능한지 미리 표시
예산이 단가보다 적으면 등록 자체를 막음

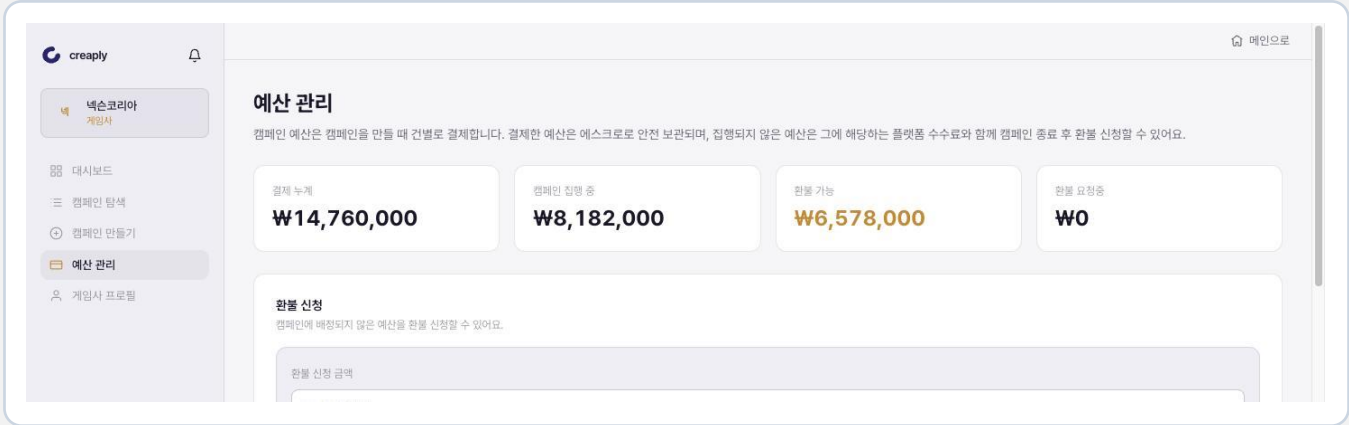
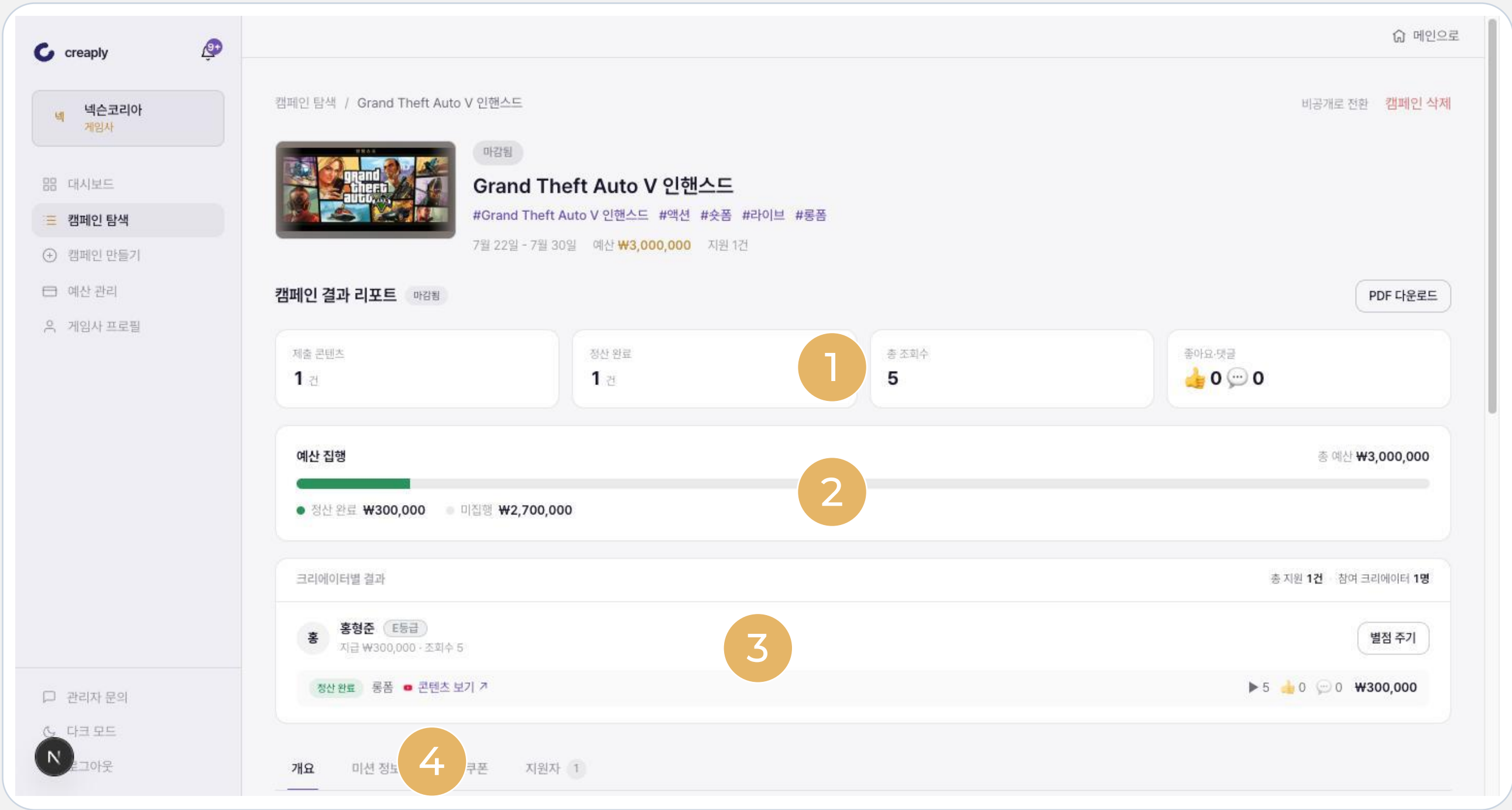
3 허용 등급 · 미션 · 쿠폰 · 테스트 계정

S~E 등급 선택, 라이브 · 룬폼 · 숏폼 미션 구성
쿠폰 · 테스트 계정은 암호화(secretBox)해 저장 후 배정 시 배분

4 결제 승인 → 캠페인 공개

웹훅과 성공 리다이렉트 두 경로를 credit_wallet_from_payment
RPC 하나로 먹등 처리, draft는 24시간 미결제 시 자동 삭제

기능 소개 03 — 게임사 · 캠페인 상세 · 검수 · 결과 리포트 · 예산 관리



예산 관리 (/studio/wallet)
결제 누계 · 집행 중(held) · 환불 가능 · 요청중
환불 신청 폼과 원장 내역. 신청은
request_studio_refund RPC가 지갑 행을 잠금

1 캠페인 결과 리포트

제출 · 정산 건수, 총 조회수 · 좋아요, 예산 집행 바
PDF 다운로드 는 @react-pdf/renderer로 서버에서 생성

2 예산 집행 현황

정산 완료 / 미집행을 확정 payout 함으로 계산
마감 시 미집행분은 자동 환불, 보너스 풀은 관리자 심사 후 환불

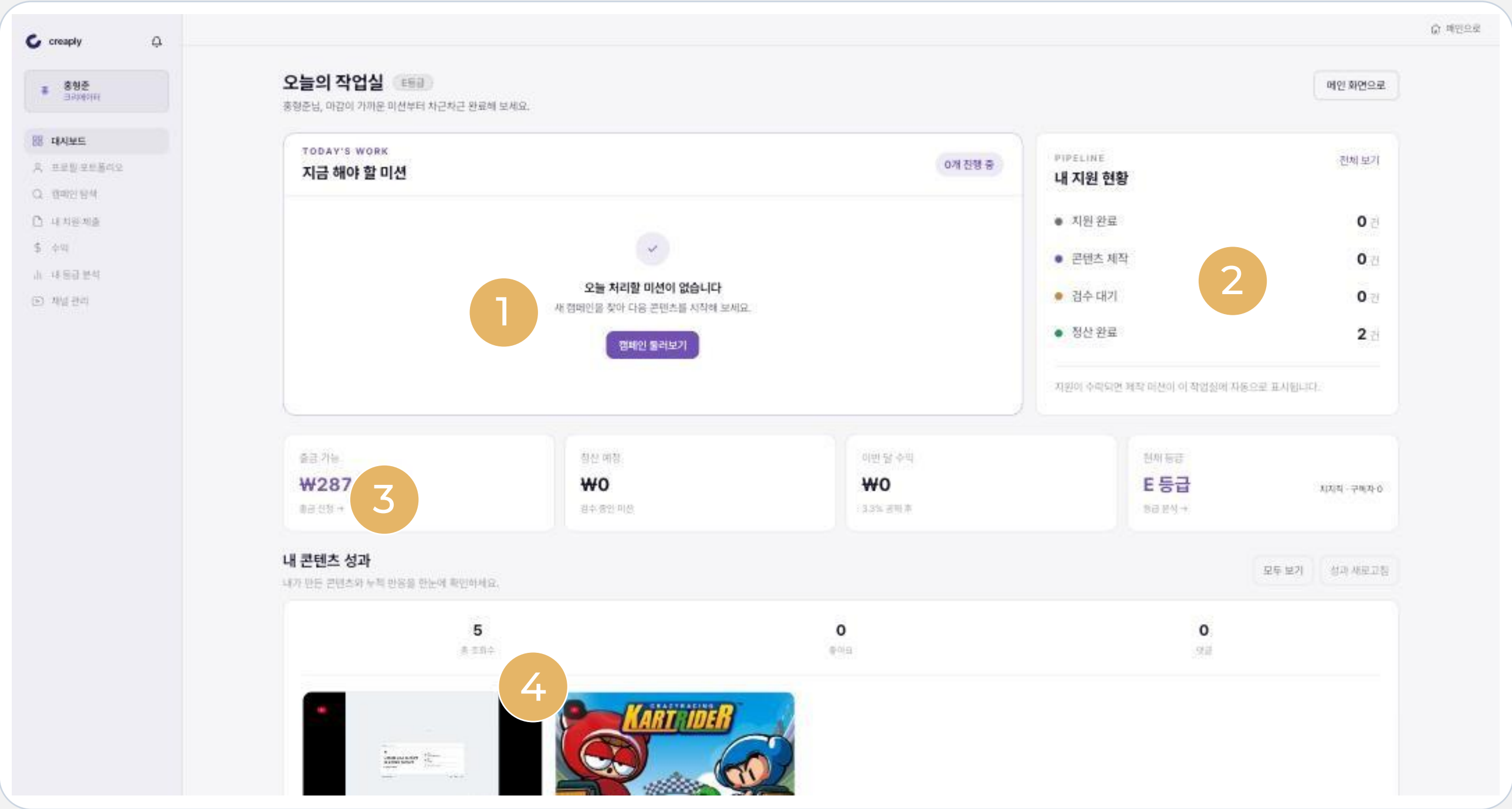
3 크리에이터별 결과 · 별점

정산 완료 배지 · 콘텐츠 링크 · 조회수, 게임사 평점(creator_reviews)
평점은 크리에이터 공개 프로필과 대시보드 모달에 반영

4 개요 · 미션 · 예산·쿠폰 · 지원자 탭

지원자 탭은 지원 / 검수 / 완료 서브탭, 제출 URL 확인 후 승인
반려는 관리자 케이스로 넘기고, 3일 무응답이면 자동 승인

기능 소개 04 — 크리에이터 · 오늘의 작업실 · 채널 인증 · 등급 분석



내 등급 분석 (/creator/grade)
현재 등급과 다음 등급까지 남은 구독자 수
채널별 산정 근거, 등급이 오르면 콘텐츠
유형별 단가가 얼마나 오르는지 표시

- 1

지금 해야 할 미션

진행 중 지원을 마감 순으로 최대 3건, 3일 이내면 강조 상태 배지(제작 중 / 검수 대기)와 실수령 예정액을 함께
- 2

내 지원 현황 파이프라인

지원 완료 → 콘텐츠 제작 → 검수 대기 → 정산 완료 카운트 applications.status 상태 머신을 그대로 보여줌
- 3

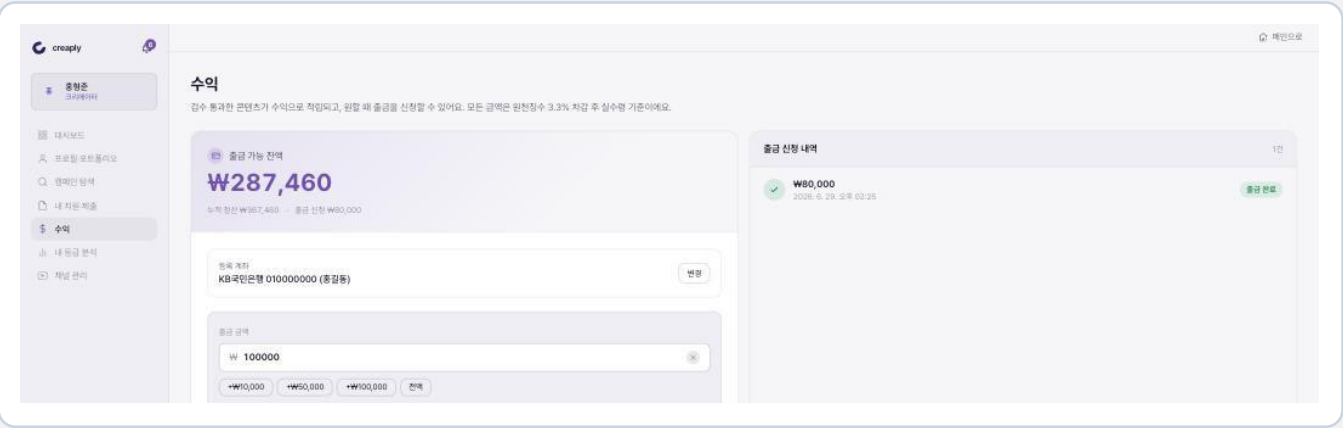
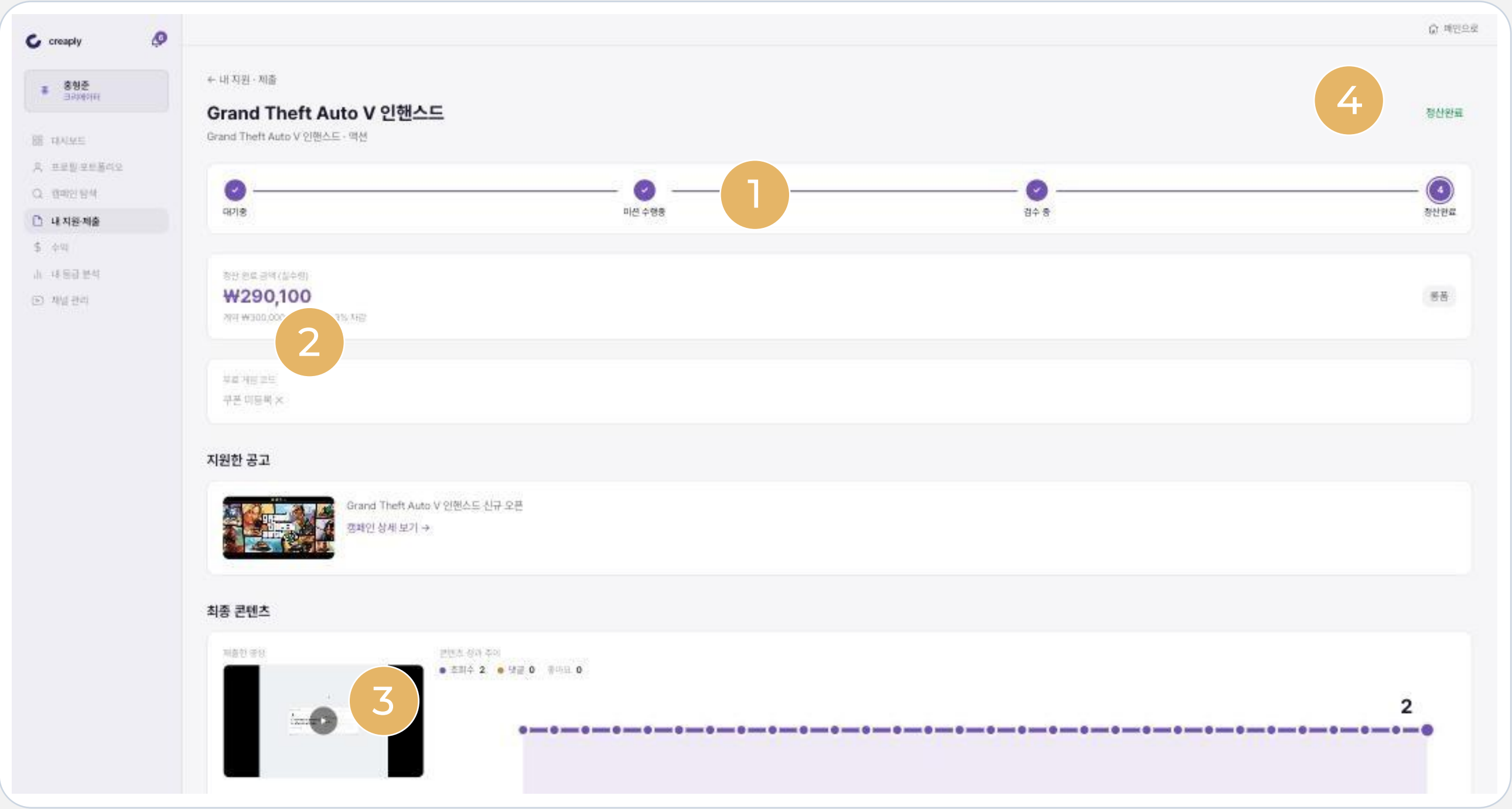
KPI 4개

출금 가능 · 정산 예정 · 이번 달 수익 · 현재 등급
등급 카드는 산정 근거(플랫폼 · 구독자 N만)와 등급 분석 링크
- 4

내 콘텐츠 성과 · 채널 인증

제출 콘텐츠 조회수 · 좋아요 · 댓글, pg_cron 6시간 수집 + 즉시 새로고침
채널은 YouTube · 치지직 · SOOP · TikTok · Instagram OAuth로 소유 인증

기능 소개 05 — 크리에이터 · 지원 · 콘텐츠 제출 · 정산 · 출금



수익 (/creator/earnings)
출금 가능 잔액 · 등록 계좌 · 출금 신청 폼
신청 내역 · 월별 정산액 차트
소득 유형(개인 / 개인사업자)별 세무 신원 등록

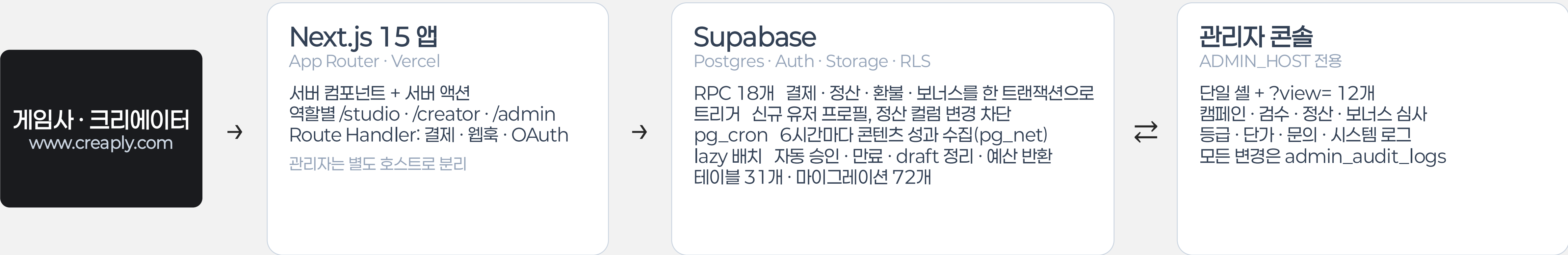
1 4단계 스테퍼
대기 → 미션 수행 → 검수 중 → 정산 완료
배정 후 5일 미제출이면 자동 만료, 예산 · 쿠폰 · 계정 회수

2 실수령 정산액
계약 ₩300,000 → 원천징수 3.3% 제외 ₩290,100
크리에이터 화면은 전부 실수령 기준으로 통일

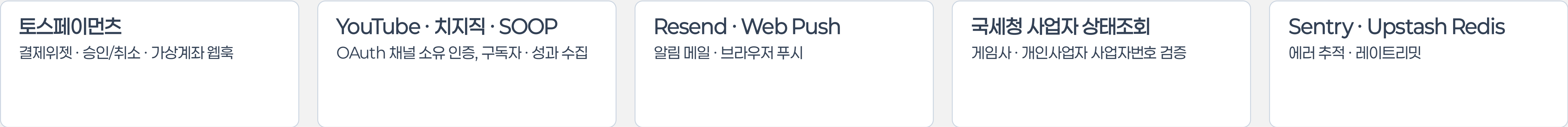
3 최종 콘텐츠 · 성과 추이
URL 제출 후 조회수 추이 차트, 게임사 평가 · 반려 사유
반려 건은 "관리자 문의"로 케이스를 열어 이의 제기

4 정산 완료 → 출금
승인 즉시 apply_application_settle RPC로 원자적 정산
지급은 관리자 수동이체 + 월별 지급명세서 CSV

Next.js 앱 하나가 역할별 대시보드 3개를 서빙하고, 돈이 오가는 로직은 전부 Postgres RPC 안에서 한 트랜잭션으로 처리한다



앱이 부르는 외부 서비스



기술 스택



결제 · 정산 · 검수 자동화까지 풀스택 단독 구현

역할별 대시보드 3개 · 화면 47개

Next.js 15 서버 컴포넌트 + 서버 액션으로 게임사 · 크리에이터 · 관리자
관리자 콘솔은 별도 호스트, 단일 셀에 12개 view

토스페이먼트 결제 · 에스프로 예산 원장

웹훅 · 리다이렉트 이중 경로를 RPC 단일 진입점으로 먹등 처리
wallets · ledger로 충전 · 홀드 · 정산 · 환불 전 내역 추적

예산 3분할(70/20/10) · 우수 콘텐츠 보너스

불변식 2겹(fee+net=budget, payout+bonus=net)으로 1원 오차 차단
마감 2단계 → 관리자 보너스 심사 → 7일 경과 시 자동 환불

선착순 배정 · 검수 · 자동 정산 상태 머신

지원 즉시 예산 상한 내 배정, 3일 무응답 자동 승인, 5일 미제출 만료
승인 시 apply_application_settle RPC로 원자적 정산

채널 소유 인증 · 등급 · 성과 자동 수집

YouTube · 치지직 · SOOP · TikTok · Instagram OAuth 소유 인증
구독자 기준 S~E 등급, pg_cron 6시간마다 조회수 · 좋아요 수집

세무 · 돈 경로 하드닝 · 테스트

주민번호 암호화 · 원천징수 3.3% · 지급명세서 CSV · 국세청 조회
결제/환불 RPC 전수 감사, Vitest 39파일 · 약 400 케이스

Why — 각 기술 선택의 근거

Q 왜 별도 백엔드 서버 없이 Next.js 서버 액션 + Supabase RPC인가?

1인 개발에서 API 서버를 따로 두면 배포 · 인증 · 스키마가 두 벌이 된다. 화면 로직은 서버 액션에 두고, 돈이 오가는 로직은 Postgres RPC(security definer + for update)에 모아 트랜잭션 경계를 DB 한 곳으로 정리했다.

Q 왜 PortOne에서 토스페이먼츠 결제위젯으로 갈아탔나?

카드사 심사와 정산 계약을 직접 맺어야 해서 PG를 한 곳으로 정리했다. 결제 승인은 웹훅과 성공 리다이렉트 두 경로로 들어오므로 credit_wallet_from_payment RPC 하나로 먹통 처리하고, 가상계좌 상태 변경도 같은 웹훅에서 받는다.

Q 왜 지갑(balance / held)과 원장(ledger)을 따로 두고 이중으로 기록하나?

마감 · 환불 · 보너스 배분이 겹치면 잔액 숫자 하나로는 어디서 들어졌는지 찾을 수 없다. 모든 이동을 ledger에 kind별로 남기고 ledger_amount_sign CHECK로 부호를 강제해, 감사 시 잔액을 원장에서 다시 계산할 수 있게 했다.

도입 전후 비교 및 최종 결과

업무	도입 전 (수기 · 이메일)	도입 후 (크레플리)	구현	효과
단가 산정	구독자 수 보고 건별 흥정	플랫폼 × 등급 × 콘텐츠 표준 단가표	platform_grade_pricing + 관리자 편집	견적 즉시 확정
결제 · 예산	선지급 후 성과 미확인	선결제 에스프로, 승인 시에만 정산	토스페이먼츠 + 지갑 · 원장 RPC	미집행분 자동 환불
크리에이터 검증	채널 소유 · 조회수 확인 불가	OAuth 소유 인증 + 성과 자동 수집	5개 플랫폼 OAuth + pg_cron 6h	실측 조회수 리포트
검수 · 정산	담당자가 수동 확인, 지연	3일 무응답 자동 승인, 원자적 정산	apply_application_settle RPC	정산 누락 · 중복 0
세무 · 지급	건별 계산, 서류 개별 수집	원천징수 3.3% · 지급명세서 CSV 일괄	withholding + 주민번호 암호화	월 1회 일괄 지급

47

역할별 화면 | 게임사 12 · 크리에이터 12 · 관리자 12
뷰

18 RPC

결제 · 정산 · 환불 · 보너스 | 전부 단일 트랜잭션

≈ 400

Vitest 테스트 케이스 | 39 파일 · 794 커밋

1. 결제 이중 충전 · 환불 TOCTOU → RPC 원자화

As-Is	웹훅과 성공 리다이렉트가 각자 "주문 확인 → 충전"을 수행, 환불 신청도 조회 → 계산 → insert가 따로 실행
Challenge	동시 요청이나 중간 실패에서 PG는 성공했는데 DB는 미반영, 가용 잔액을 초과한 환불 신청이 가능
To-Be	security definer RPC 3종이 주문 · 지갑 · 원장을 for update로 잠가 한 트랜잭션으로 처리. 로컬 Postgres에서 동시 호출을 재현해 1회만 충전됨을 검증

2. 예산 2분할 → 3분할(70/20/10)과 보너스 풀

As-Is	net_budget 전액이 모집 상한, 우수 콘텐츠 보너스 재원이 없음
Challenge	기존 환불 · 정산 수식이 $fee + net = budget$ 에 기대고 있어 건드리면 결제 경로 전체가 흔들림, 반올림 1원 잔류
To-Be	net 아래를 payout / bonus로만 쪼개 불변식 2겹 유지. 마감 2단계(지금 풀 환불 → 관리자 심사 → 7일 자동 환불), 반올림은 2회만 하고 나머지는 뽀샵

3. 돈 경로 가드 하드닝 (감사 후속)

As-Is	음수 배분이 잔액을 늘리고, amount 키 누락 NULL이 검증을 통과하고, greatest()가 NULL 인자를 무시
Challenge	같은 유형의 결함이 작업마다 반복됨. 호출부마다 고치면 다음 기능에서 또 새는 구조
To-Be	결제 · 환불 RPC 전수 감사 후 NULL 가드와 ledger_amount_sign CHECK를 DB에 추가, 미사용 apply_campaign_hold 권한 제거

4. 마이그레이션 충돌 · 감사로그 표류

As-Is	순번 방식 마이그레이션이 브랜치 병렬 작업에서 번호가 겹치고, admin_audit_logs는 운영 DB에만 수동 생성
Challenge	누락된 테이블 · 마이그레이션이 로컬 재생에서만 드러나 운영과 코드가 조용히 어긋남
To-Be	YYYYMMDDHHMMSS 규칙 + migrationSafety 테스트, 실제 insert/select에서 스키마를 역추적해 if not exists로 이관

프로젝트 회고

이 프로젝트를 시작할 때의 목표는 단순했다.

“게임사와 크리에이터를 표준화된 단가로 연결하자.”

하지만 실제 개발을 진행하며 깨달은 것은, 화면이나 기능을 구현하는 일보다 **결제·배정·검수·정산·환불이 서로 어긋나지 않도록 하나의 일관된 자금 흐름을 설계하는 일**이 훨씬 어렵다는 점이었다. 각각의 기능이 개별적으로 정상 작동하는 것만으로는 충분하지 않았다. 전체 과정에서 데이터와 금액이 정확하게 이어지고, 예외 상황에서도 문제가 발생하지 않도록 설계해야 했다.

요구사항이 구체적으로 정리되지 않은 상태에서 “이런 기능이 필요하다”, “화면이 조금 어색한 것 같다”와 같은 피드백을 받는 경우도 많았다. 그때마다 내가 직접 요구사항을 다시 정의하고, 페이지별 API와 데이터 흐름을 확인하며, QA까지 혼자 수행해야 했다. 이 과정을 통해 개발자는 단순히 주어진 기능을 구현하는 사람을 넘어, 모호한 요구사항을 구체적인 정책과 기능으로 바꾸는 역할도 해야 한다는 것을 체감했다. 동시에 기획·개발·QA를 혼자 병행하는 일이 생각보다 훨씬 어렵다는 점도 알게 되었다.

Claude Code를 활용하면서 구현 속도 자체는 크게 높일 수 있었다. 다만 하나의 세션 안에서 여러 Subagent에게 기능을 나누어 맡기는 방식보다, **성격이 명확히 다른 기능을 각각 별도의 세션에서 진행하는 방식**이 더 효율적이었다. 기능별 작업 맥락이 섞이지 않아 결과물을 관리하기 쉬웠고, 현재 진행 중인 작업과 남아 있는 작업도 더 명확하게 파악할 수 있었다.

깨달은 점

개발 과정에서 반복되는 작업을 자동화하는 것이 생산성 향상에 큰 도움이 되었다. 대표적으로 브라우저에서 기능을 직접 확인하는 QA 절차를 SKILL.md로 만들어 두고, 새로운 기능이 추가될 때마다 LLM이 동일한 기준으로 검증하도록 했다. 반복적인 확인 작업을 표준화하자 누락이 줄었고, 전체적인 개발 속도도 눈에 띄게 향상되었다.

반면 PG 심사, 플랫폼별 OAuth 권한 인증, 네이버 서치어드바이저와 Google Search Console 등록 같은 외부 절차는 예상보다 훨씬 많은 시간이 필요했다. 이용약관, 개인정보처리방침, 환불정책 등의 정책 문서를 실제 서비스 구조와 일치하도록 만들고, 그 정책에 맞게 기능을 구현하는 과정 역시 단순히 AI를 활용해 빠르게 처리할 수 있는 일이 아니었다. 정확한 구현을 위해서는 충분한 사전 조사와 정책 설계가 반드시 필요했다.

플랫폼별 크리에이터 등급과 콘텐츠 단가를 설계하는 과정에서도 비슷한 점을 느꼈다. 기술적으로 기능을 구현할 수 있는 것과 실제 시장에서 운영 가능한 서비스를 만드는 것은 전혀 다른 문제였다. 적절한 등급 기준과 단가를 정하려면 크리에이터 시장, 콘텐츠 유형, 광고 효율, 게임사의 예산 구조 등을 함께 이해해야 했다.

결국 이번 프로젝트를 통해 개발자는 개발 기술만 알아서는 충분하지 않다는 점을 배웠다. 좋은 서비스를 만들기 위해서는 **서비스가 속한 산업과 비즈니스 구조를 이해하고, 모호한 요구사항을 구체적인 정책으로 정의하며, 반복되는 업무를 자동화할 수 있어야 한다**. 이번 프로젝트의 경우에는 개발 역량뿐만 아니라 게임과 마케팅 도메인에 대한 이해가 서비스의 완성도를 결정하는 중요한 요소였다.

PROJECT.2

게임 현지화 번역 · 협업 검수 워크스페이스

02



프로젝트 기간	2026.09.16 ~ 2026.09.28
프로젝트 현황	운영 중 (사내 배포)
팀 구성	개인 프로젝트
역할	풀스택 개발 · 배포
도메인	https://translation.theplayplus.com
기술 스택	Spring Boot 3 · React 19 · PostgreSQL 16
배포	AWS Lightsail · Docker · nginx (HTTPS)

Why — 게임 퍼블리셔의 중국어 게임 텍스트 현지화 검수 문제

5만 행 엑셀을 여러 명이 나눠 받아 메일로 주고받던 번역 검수

#	ID	간체 원문	영어	기존 한국어	수정 한국어 (Fix)	1차	2차	확인 필요	비고
2987	Building	守护古树 ×2		고목 수호	고목 수호	☐	☐	☐	
3082	Building	霜冻冰原 ×5		프로스트 아이스필드	프로스트 아이스필드 길이	☐	☐	☐	
3850	Building	异蔬果林 ×9		과일과 채소의 숲	열매과 채소의 숲 길이	☐	☐	☐	
3851	Building	梦境丛林 ×7		드림정글	드림정글	☐	☐	☐	
9502	Dungeon	苹果战将 ×313		사과 전사	사과 전사	☐	☐	☐	
9504	Dunaeon	吞噬菜王 ×3		포식 채소왕	포식 채소왕 길이	☐	☐	☐	

표기가 갈리는 번역

같은 원문 313곳이 행마다 다르게 번역됨
영웅 이름·UI 용어가 검수자마다 달라
게임 안에서 표기가 갈림

How + Result : 단어장 + 동일 원문 자동 통일

추적 안 되는 검수

엑셀 복사본을 나눠 보내고 메일로 회수
누가 무엇을 고쳤는지, 1차·2차가
어디까지 됐는지 알 수 없음

How + Result : 동시 편집 그리드 + 1차/2차 체크

빌드해야 보이는 UI 깨짐

한국어가 길어져 버튼·툴팁이 잘림
빌드해 봐야 발견되고, 5만 행을
사람이 길이까지 볼 수는 없음

How + Result : 길이 플래그 + GPT 짧게 줄이기

번역 검수 워크스페이스

게임 단위로 xlsx를 올려 열 역할만 지정하면, 엑셀식 그리드에서 여러 명이 함께 검수하고 원본 형식 그대로 내려받는 흐름



주요 화면 · 기능

- WS-01 게임 목록 · 누적 GPT 비용
- WS-02 파일 · 폴더 업로드 (xlsx · CSV) · 한 문장 빠른 번역
- WS-03 정규화 (열 역할 드롭다운 · Fix 열 선택 · 열 쌍 여러 개)
- WS-04 검수 그리드 (가상 스크롤 · 키보드 편집 · 붙여넣기 분배)
- WS-05 필터 10종 · 검색 · 50/100행 쪽 넘기기
- WS-06 1차 · 2차 · 확인 필요 · 비고 · 보기 단위 일괄 체크

- AI-01 단어장 (게임별 · 공유 승격 · 게임 전체 일괄 적용)
- AI-02 동일 원문 자동 통일 · ×N 모아 보기 · 불일치 필터
- AI-03 우클릭 GPT 번역 · 짧게 줄이기 (후보 고르기)
- AI-04 길이 초과 · 용어 불일치 플래그
- AI-05 실시간 SSE 동기화 · 접속자 표시 · 수정 이력
- AI-06 내보내기 (원본 서식 유지 · 비고 열 · CSV)

기능 소개 01 — 게임 · 업로드 · 정규화 (열 역할 지정)

숲의 소환사 > 配置文本.xlsx

Sheet1 · 49,565행

1

헤더 행 1

	A	B	C
	ID	간체 원문	기존 한국어
헤더	配置表名	Chinese String	Korean String
1	ActivityDesc	1.活动期间烹饪<color=#78fa5a>限...	1. 이벤트 기간 동안 <color=#78fa5a...
2	ActivityDesc	1.活动期间, 根据<color=#78fa5a>...	1. 이벤트 기간 동안 <color=#78fa5a...
3	ActivityFund	收集 <color=#78fa5a>月饼</color>...	<color=#78fa5a>월병</color>을 ...
4	ActivityFund	活动期间, 达成<color=#78fa5a>...	이벤트 기간 동안 <color=#78fa5a>...
5	ActivityFund	收集 <color=#df1311>贝壳</color>...	<color=#df1311>조개껍데기</color>...

+ 원문·한국어 쌍 추가

수정 한국어(Fix) 쓸 곳 ☐ 새 열로 추가 ☒ 기존 한국어 열에 덮어쓰기

49565행 검수 대상 · 건너뛴 0 · 기존 한국어 49565행

이 시트 정규화

이 게임의 파일					파일 이름을 누르면 원본을 내려받을 수 있습니다	
원본 파일	시트	행	단계	올린 날짜		
· 配置文本.xlsx	1	49,566	검수 중	2026-09-28	이어서 진행	삭제
· UI文本_KR_revised_1차완료_0925.xlsx	1	6,045	검수 중	2026-09-27	이어서 진행	삭제
· 代码文本_KR_revised_1차완료_0925.xlsx	1	3,599	정규화 전	2026-09-27	이어서 진행	삭제

게임 안의 파일 목록 · 단계(정규화 전 / 검수 중)
이어서 진행 → 정규화 또는 검수 그리드로

- 1

게임 단위 공유

게임을 만들고 그 안에 xlsx · CSV 파일이나 폴더째 업로드.
같은 게임끼리 단어장 · 번역 메모리 · 검수 결정을 공유
- 2

열 역할 드롭다운

헤더 행을 고르면 열 이름(CN/EN/KR · 中文/韩语)으로 역할 추측.
ID · 간체 원문 · 영어 · 기존 한국어를 시트마다 지정,
원문 열이 여러 개면 열 쌍을 여러 개 두고 한 파일로 처리
- 3

Fix 쓸 곳 선택

새 열 추가 / 기존 한국어 열에 덮어쓰기.
검수 화면에는 둘 다 "기존 한국어"와 "Fix"로 따로 보이고
내보낼 때만 달라짐
- 4

정규화 → 세그먼트

빈 원문 · N/A 표시 행은 건너뛰고 칸 단위 세그먼트 생성.
같은 게임의 이전 결정(Fix · 체크)을 새 파일이 이어받음

기능 소개 02 — 엑셀식 검수 그리드 (시트 탭 · 필터 · 셀 편집 · 길이 플래그 · 체크)

숲의 소환사 > 配置文本.xlsx 정규화

● 접속 1

● 홍형준

단어장

내보내기

Sheet1 (49,565)

불일치 다시 계산 보기

100행

1 / 496

1~100 / 49,565행

전체

ID · 원문 · Fix 검색

3행 · 수정 한국어 (Fix)

1. 이벤트 기간 동안 <color=#78fa5a>음식 섭취 횟수</color>를 기준으로 랭킹이 집계됩니다.\n2. 요리 대회 한정 음식과 일부 음식은 일일 섭취 횟수 한도를 소모하지 않습니다.\n3. 랭킹 이벤트 종료 후 보상이 정산되어 지급됩니다.

#	ID	간체 원문	영어	기준 한국어	수정 한국어 (Fix)	1차	확인 필요	비고
2	ActivityDesc	1.活动期间烹饪<color=#78fa5a>限...		1. 이벤트 기간 동안 <color=#78fa5a>한정 음식	1. 이벤트 기간 동안 <color=#78fa5a>한정 음식	✓	✓	
3	ActivityDesc	1.活动期间, 根据<color=#78fa5a>...		1. 이벤트 기간 동안 <color=#78fa5a>음식 섭취 횟	1. 이벤트 기간 동안 <color=#78fa5a>음식 섭취 횟	✓	✓	
4	ActivityFund	收集 <color=#78fa5a>月饼</color>...		<color=#78fa5a>월병</color>을 모...	<color=#78fa5a>월병</color>을 모으면 희귀	✓	✓	
5	ActivityFund	活动期间, 达成<color=#78fa5a>美...		이벤트 기간 동안 <color=#78fa5a>음식 섭취 횟	이벤트 기간 동안 <color=#78fa5a>음식 섭취 횟	✓	✓	
6	ActivityFund	收集 <color=#df1311>贝壳</color>...		<color=#df1311>조개껍데기</color>...	<color=#df1311>조개껍데기</color>를 모으면			
7	ActivityFund	活动期间, 使用<color=#78fa5a>殷...		이벤트 기간 동안 <color=#78fa5a>주사위 사용	이벤트 기간 동안 <color=#78fa5a>주사위 사용			
8	ActivityFund	活动期间, 使用<color=#78fa5a>英...		이벤트 기간 동안 <color=#78fa5a>영웅 모집권	이벤트 기간 동안 <color=#78fa5a>영웅 모집권			
9	ActivityFund	活动期间, 使用<color=#78fa5a>灵...		이벤트 기간 동안 <color=#78fa5a>영수 알 사용	이벤트 기간 동안 <color=#78fa5a>영수 알 사용			
10	ActivityFund	活动期间, 获得<color=#78fa5a>木...		이벤트 기간 동안 <color=#78fa5a>목정광-파랑	이벤트 기간 동안 <color=#78fa5a>목정광-파랑			
11	ActivityFund	活动期间, 使用<color=#78fa5a>鱼...		이벤트 기간 동안 <color=#78fa5a>미끼 사용 수	이벤트 기간 동안 <color=#78fa5a>미끼 사용 수			
12	ActivityFund	活动期间, 提升<color=#78fa5a>战...		이벤트 기간 동안 <color=#78fa5a>전투력</color>	이벤트 기간 동안 <color=#78fa5a>전투력</color>			
13	ActivityFund	活动期间, 消耗<color=#78fa5a>钻...		이벤트 기간 동안 <color=#78fa5a>다이아 소모	이벤트 기간 동안 <color=#78fa5a>다이아 소모			
14	ActivityFund	收集 <color=#df1311>恶魔南瓜</c...		<color=#df1311>악마 호박</color>...	<color=#df1311>악마 호박</color>을 모으면 희귀			
15	ActivityFund	活动期间, 获得<color=#78fa5a>森...		이벤트 기간 동안 <color=#78fa5a>숲의 이슬 획	이벤트 기간 동안 <color=#78fa5a>숲의 이슬 획			
16	ActivityFund	活动期间, 使用<color=#78fa5a>殷...		이벤트 기간 동안 <color=#78fa5a>주사위 사용	이벤트 기간 동안 <color=#78fa5a>주사위 사용			
17	ActivityFund	活动期间, 使用<color=#78fa5a>殷...		이벤트 기간 동안 <color=#78fa5a>주사위 사용	이벤트 기간 동안 <color=#78fa5a>주사위 사용			
18	ActivityFund	活动期间, 使用<color=#78fa5a>殷...		이벤트 기간 동안 <color=#78fa5a>주사위 사용	이벤트 기간 동안 <color=#78fa5a>주사위 사용			
19	ActivityFund	活动期间, 使用<color=#78fa5a>殷...		이벤트 기간 동안 <color=#78fa5a>주사위 사용	이벤트 기간 동안 <color=#78fa5a>주사위 사용			
20	ActivityForest...	玩家x获得x晶果, 增加全体属性		플레이어 x님이 x 수정열매를 획득하여 ...	플레이어 x님이 x 수정열매를 획득하여 전체 능력치			
21	ActivityForest...	玩家x击败x级别BOSS, 全队玩家进...		플레이어 x님이 x레벨 BOSS를 처치하...	플레이어 x님이 x레벨 BOSS를 처치하여 팀 전원0			
22	ActivitySnowma...	雪人队		눈사람 팀	눈사람 팀			

1 시트 탭 · 쪽 넘기기

시트별 행 수, 전체 / 50 / 100행 보기
496쪽을 번호로 바로 점프, 행 높이 고정 가상 스크롤

2 필터 10종 · 검색

수정됨 · 1차/2차 안 함 · 확인 필요 · 용어 불일치 · Fix 비어 있음 ...
ID · 원문 · Fix 검색, 조건은 URL 쿼리로 공유

3 편집 줄

고른 칸의 전체 내용을 표 위에 펼쳐 보고 바로 고침
<color> 태그 · \n 자리표시자를 그대로 보존

4 셀 편집 · 키보드

Enter 편집 · Tab 다음 행 · ⌨Enter 1차 체크
여러 줄 붙여넣기는 아래 행으로 분배, blur 시 자동 저장

5 길이 플래그 · 변경 표시

한국어 ÷ 원문 글자 수 비율 초과면 "길이" 배지, Fix가 기존과 다르면
노란 배경

6 1차 · 2차 · 확인 필요 · 비교

행마다 저장, 상태바에 수정 915 · 1차 9 · 2차 7 집계

기능 소개 03 — 단어장 · 동일 원문 자동 통일 · 우클릭 GPT

● 접속 1 ● 홍형준 ▾

단어장

내보내기

1

이번 게임 75

공유 단어집 0

2

찾을 문자열·한국어·비고 검색

기존 한국어	수정 한국어 (Fix)	1차	2차
1. 이벤트 기간 동안 <color=#78fa5a>한정 음식</color> 길이	1. 이벤트 기간 동안 <color=#78fa5a>한정 음식</color> 길이	✓	✓
1. 이벤트 기간 동안 <color=#78fa5a>음식 섭취 길이	1. 이벤트 기간 동안 <color=#78fa5a>음식 섭취 길이	✓	✓
<color=#78fa5a>월병</color>을 모...</color>을 모으면 희귀 길이	<color=#78fa5a>월병</color>을 모으면 희귀 길이	✓	✓
이벤트 기간 동안 <color=#78fa5a>음식 섭취 횟 길이	이벤트 기간 동안 <color=#78fa5a>음식 섭취 횟 길이	✓	✓
<color=#df1311>조개껍데기</color>를 모으면 길이	<color=#df1311>조개껍데기</color>를 모으면 길이	□	□
이벤트 기간 동안 <color=#78fa5a>주사위 사용 길이	이벤트 기간 동안 <color=#78fa5a>주사위 사용 길이	□	□
이벤트 기간 동안 <color=#78fa5a>영웅 모집권 길이	이벤트 기간 동안 <color=#78fa5a>영웅 모집권 길이	□	□
이벤트 기간 동안 <color=#78fa5a>영수 알 사용 길이	이벤트 기간 동안 <color=#78fa5a>영수 알 사용 길이	□	□
이벤트 기간 동안 <color=#78fa5a>목정광-파랑 길이	이벤트 기간 동안 <color=#78fa5a>목정광-파랑 길이	□	□
이벤트 기간 동안 <color=#78fa5a>미끼 사용 수 길이	이벤트 기간 동안 <color=#78fa5a>미끼 사용 수 길이	□	□
이벤트 기간 동안 <color=#78fa5a>전투력</color> 길이	이벤트 기간 동안 <color=#78fa5a>전투력</color> 길이	□	□

가치 있는 물품 → 특급

수정 삭제 공유로 올리기

한→한 들어 있으면

巨盾守卫·蕾欧娜 → 대방패 수호자: 발테라

수정 삭제 공유로 올리기

중→한 통째로

건너뛰기 → 스킵

수정 삭제 공유로 올리기

한→한 들어 있으면

犬弓·旺财 → 강아지 사수: 왕차이

수정 삭제 공유로 올리기

중→한 통째로

棍术宗师·大宝 → 봉술 판다: 디바오

수정 삭제 공유로 올리기

술의 소환사 > 配置文本.xlsx 정규화

● 접속 1 ● 홍형준 ▾

단어장

내보내기

Sheet1 (49,565)

불일치 다시 계산

보기 전체 50행 100행

1 / 1

1-1 / 1행

전체

企鵝队

같은 원문 '企鵝队' 모아 보는 중 · 1행 × 전체 보기
브라우저 뒤로 가기로도 돌아갑니다

칸을 고르면 전체 내용이 여기 보입니다

#	ID	간체 원문	영어	기존 한국어	수정 한국어 (Fix)	1차	2차	확인 필요	비고
23	ActivitySnowma...	企鵝队 ×4		펭귄 팀	펭귄 팀	□	□	□	

1. 이벤트 기간 동안 <color=#78fa5a>음식 섭취 길이

<color=#78fa5a>월병

이벤트 기간 동안 <colo

단어장에 추가 (한국어)

짧게 줄이기

4

5

- 1

게임별 단어장 · 전체 다시 적용

중→한 / 한→한 규칙, 통째로 · 들어 있으면 매칭. 게임 전체 Fix에 일괄 적용
- 2

단어장 항목 · 공유 승격

항목마다 수정 · 삭제 · 공유로 올리기. 공유 단어집은 다른 게임에서도 적용
원문 · Fix 칸 우클릭 → "단어장에 추가"로 셀에서 바로 등록
- 3

동일 원문 자동 통일

Fix 저장 시 같은 원문 행을 게임 전체에서 한 트랜잭션으로 함께 수정
×N 칩 클릭으로 같은 원문만 모아 보기, 뒤로 가기로 복귀
- 4

짧게 줄이기 (GPT)

목표 글자 수를 정하면 GPT 후보 여러 개 → 골라서 Fix에 넣음
길이 플래그 행을 검수 단계에서 바로 해결
- 5

GPT 번역 · 불일치 필터

단어장 · 번역 메모리를 붙여 행 단위 번역. 단어장과 어긋나면 용어 불일치,
같은 원문인데 Fix가 다르면 동일 원문 불일치 필터로 모아 봄

기능 소개 04 — 실시간 협업 · 일괄 체크 · 필터 · 내보내기

1

● 접속 1

● 홍형준 ▾

단어장

내보내기

전체 보기

전체 50행 100행

« < 1 / 496 > »

1-100 / 49,565행

전체 ▾

ID · 원문 · Fix 검색

기존 한국어	수정 한국어 (Fix)	1차	2차	확인 필요	비고
1. 이벤트 기간 동안 <color=#78fa5a...	1. 이벤트 기간 동안 <color=#78fa5a>한정 음식 길이				
1. 이벤트 기간 동안 <color=#78fa5a...	1. 이벤트 기간 동안 <color=#78fa5a>음식 섭취 길이				
<color=#78fa5a>월병</color>을 모...	<color=#78fa5a>월병</color>을 모으 길이				
이벤트 기간 동안 <color=#78fa5a>...	이벤트 기간 동안 <color=#78fa5a>음식 섭취 횟 길이	✓	✓		

이 보기의 100행 — 1차

모두 체크

모두 해제

취소

Sheet1 (49,565)

불일치 다시 계산

보기 전체 50행 100행

« < 1

1-100 / 368행

동일 원문 불일치 ▾

ID · 원문 · Fix 검색

칸을 고르면 전체 내용이 여기 보입니다

#	ID	간체 원문	영어	기존 한국어	수정 한국어 (Fix)	1차	2차	확인 필요	비고
2987	Building	守护古树 x2		고목 수호	고목 수호	☐	☐	☐	
3082	Building	霜冻冰原 x5		프로스트 아이스필드	프로스트 아이스필드 길이	☐	☐	☐	
3850	Building	异蔬果林 x9		과일과 채소의 숲	열매과 채소의 숲 길이	☐	☐	☐	
3851	Building	梦境丛林 x7		드림정글	드림정글	☐	☐	☐	
9502	Dungeon	苹果战将 x313		사과 전사	사과 전사	☐	☐	☐	
9504	Dunaeon	吞噬菜王 x3		포식 채소왕	포식 채소왕 길이	☐	☐	☐	

1 실시간 동기화 · 접속자

프로젝트당 SSE 스트림 하나로 다른 사람이 고친 셀 · 체크 · 비교가 바로 반영.
로그인 없이 이름만 입력(X-Reviewer-Id), 접속자 수 표시,
수정 이력에 누가 언제 고쳤는지 기록

2 보기 단위 일괄 체크

1차 · 2차 · 확인 필요 머리글 더블클릭 → 지금 보기의 행 모두 체크/해제.
1,000행 넘으면 서버 작업으로 돌리고 SSE로 진행 알림

3 문제 행만 모아 보기

동일 원문 불일치 368행처럼 필터 하나로 문제 행만.
시트별 수정됨 · 1차 · 2차 건수는 집계 쿼리 한 번으로

4 검색 · 내보내기

ID · 원문 · Fix 검색. 내보내기는 원본 워크북을 POI로 열어
지정 열에만 Fix를 쓰고 서식 · 다른 시트 보존, 비교 열 · CSV(BOM)

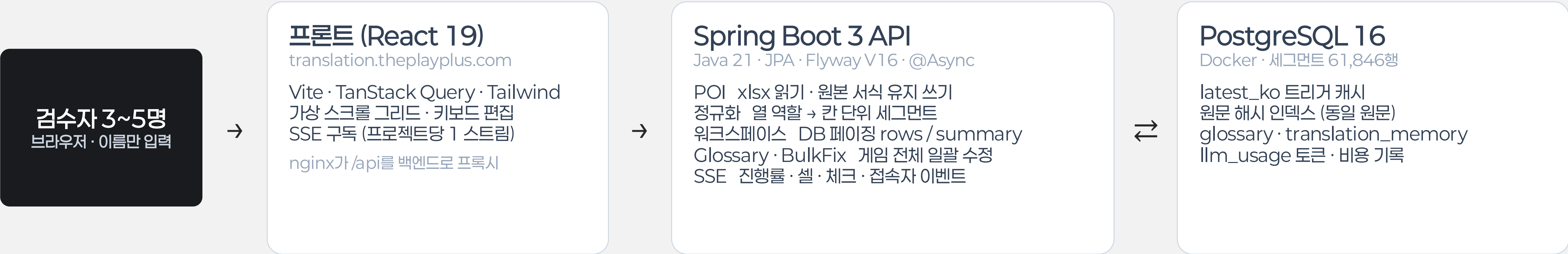
Sheet1 (49,565)

불일치

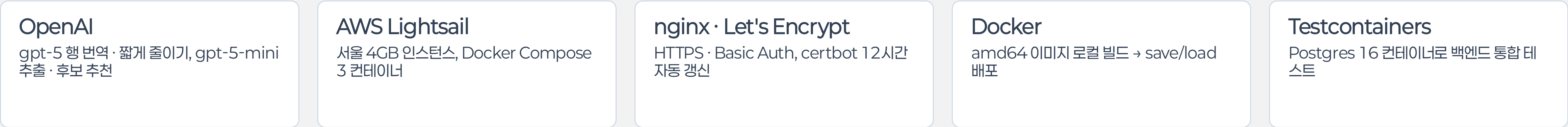
3744행 · 수정 한국어 (Fix) 왕행권의 보물

#	ID	간체 원문	영어	기존 한국어
23	ActivitySnowma...	企鹅队 x4		펭귄 팀
3744	Building	企鹅王的宝藏		왕행권의 보물
9513	Dungeon	企鹅王 x17		펭귄왕
9531	Dungeon	企鹅王子 x4		프린스 펭귄
9535	Dungeon	企鹅王 x17		펭귄왕
9551	Dunaeon	企鹅王子 x4		프린스 펭귄

React 그리드가 Spring Boot API를 부르고, 5만 행 페이징 · 동일 원문 통일 · 불일치 계산은 Postgres 안에서 끝낸다



백엔드가 부르는 외부 서비스 · 인프라



기술 스택



검수 그리드 · 협업 · 배포 구현

엑셀식 검수 그리드

가상 스크롤 · 키보드 이동 · 붙여넣기 분배 · 셀 우클릭 메뉴
5만 행 시트를 DB 페이징으로 100행씩, 요약은 집계 쿼리로
필터 10종 · 검색 · 시트 탭, 조건은 URL 쿼리에 보관

협업 검수 · 실시간

이름만으로 검토자, 1차 · 2차 · 확인 필요 · 비교, 보기 단위 일괄 체크
프로젝트당 SSE 1개로 셀 · 체크 · 접속자 동기화, 요청이 소유한 열만 UPDATE

XLSX 정규화 · 내보내기

열 역할 추측 · 열 쌍 여러 개 · CSV 인코딩 자동 판별(UTF-8 / MS949 / GB18030)
POI로 원본 서식 유지, 비교 열, 리비전 이력 시트

단어장 · 동일 원문 통일

게임별 / 공유 단어장, 통째로 · 부분 매칭, 게임 전체 일괄 적용
원문 해시 인덱스로 같은 원문 행을 한 트랜잭션에 통일

GPT 번역 파이프라인

묵음 번역 · 자리표시자/태그 검증 · 용어 누락 재시도 · 짧게 줄이기
번역 메모리 0차 재사용, 유사 문장 참고, 토큰 · 비용 기록

배포 · 운영

Docker Compose 3 컨테이너, Lightsail 4GB, HTTPS 자동 갱신
운영 DB · 파일 복사 스크립트로 로컬에서 재현한 뒤 배포

Why — 각 기술 선택의 근거

Q 왜 그리드 라이브러리 대신 엑셀식 그리드를 직접 만들었나?

검수자가 쓰던 건 엑셀이라 Enter · Tab · 여러 줄 붙여넣기 분배 · ⌘Enter 체크 같은 손버릇을 그대로 살려야 했음.
5만 행은 화면에 다 그리지 않고 100행씩 DB에서 페이징, 행 높이 고정 가상 스크롤로 쪽 이동이 즉시

Q 왜 현재 번역(latest_ko)을 Java가 아니라 Postgres 트리거로 유지하나?

리비전을 쓰는 경로가 7곳이라 한 곳만 빠뜨려도 목록이 옛 값을 보여 줌.
translation_revision 트리거가 segment.latest_ko를 갱신하니 필터 · 정렬 · 내보내기가 조인 없이 한 열만 읽음

Q 왜 로그인 대신 이름만 받나?

사내 3~5명이 쓰는 도구라 계정 · 비밀번호 관리가 도입 장벽. 외부 접근은 nginx Basic Auth + HTTPS로만 막고
검토자 이름은 X-Reviewer-Id 헤더로 실어 수정 이력 · 담당 · 접속자 표시에만 씀 (행 잠금 없음, 마지막 저장 우선)

도입 전후 비교 및 최종 결과

업무	도입 전 (엑셀 · 메일)	도입 후 (검수 워크스페이스)	구현	효과
검수 분배	엑셀 복사본을 나눠 보내고 메일로 회수	한 URL에서 동시에 검수, 행 단위 1차 · 2차 체크	SSE + 부분 UPDATE(@DynamicUpdate)	합치는 작업 0
용어 통일	검수자마다 다른 표기, 뒤늦게 발견	단어장 일괄 적용 + 같은 원문 자동 통일	GlossaryApplier · 원문 해시 인덱스	불일치 필터로 즉시 확인
UI 길이	빌드 뒤 잘림 발견	비율 초과 행 "길이" 배지 + GPT 짧게 줄이기	글자 수 비율 계산 · ShortenPopover	검수 단계에서 차단
5만 행 열기	목록 API가 쪽마다 전체 로드 (2.3초 · 620MB)	시트 안 100행 DB 페이징, 요약은 집계 쿼리	latest_ko 트리거 캐시 + OFFSET 페이징	쪽 이동 즉시
초벌 번역	번역가가 처음부터 작성	GPT 묶음 번역 + 번역 메모리 0차 재사용	OpenAI JSON 모드 · 자리표시자 검증	6만 행에 GPT 비용 \$2.66

49,566

한 시트 행 수 | 100행 DB 페이징 · 즉시 이동

563

테스트 케이스 | 백엔드 355 · 프론트 208

13일

설계 → 운영 배포 | 204 커밋 · V16

1. 검수 목록이 프로젝트 전체를 메모리에 올림 → DB 페이징

As-Is	목록 API가 쪽마다 5만 행 + 10만 리비전을 다 읽어 통계를 내고 한 쪽만 잘라 냄. 쪽당 2.3초 · 620MB, 1GB 서버에서는 빈 화면
Challenge	현재 번역이 "리비전 최댓값"이라 정렬 · 필터마다 조인이 필요하고, 시트별 요약도 전체를 훑어야 함
To-Be	segment.latest_ko를 트리거로 유지하고 시트 안에서 OFFSET 페이징, 요약은 집계 쿼리로 분리. V14 백필 61,826행 2.8초, 쪽 이동 즉시

2. 동시 편집이 서로의 저장을 되돌림 → 소유 열만 UPDATE

As-Is	두 사람이 같은 행의 Fix와 담당을 고치면 나중 저장이 앞 저장을 지움 (JPA save()가 엔티티 전체 열을 UPDATE)
Challenge	잠금이나 충돌 대화상자 없이(제품 결정) 서로의 수정을 지켜야 하고, 실시간 반영은 바뀐 열만 전파해야 함
To-Be	@DynamicUpdate + 부분 @Modifying 쿼리로 요청이 소유한 열만 쓰고, 커밋 뒤 SSE로 변경 열만 전파. "detached 엔티티를 save()하지 않는다"를 규칙으로

3. 용어 불일치 다시 계산 10분 → 버튼 막고 증분 계산

As-Is	단어장을 저장한 뒤 5만 행 시트의 불일치를 다시 계산하면 약 10분, 옛 용어표 기준 표시 4,950건이 함께 지워짐
Challenge	바뀐 행마다 bulk update가 영속성 컨텍스트 전체 flush를 강제해 제공 비용, 옛 용어표와 단어장 두 기준이 섞임
To-Be	전체 재계산 버튼을 막고(툴팁으로 안내) 저장 시점에 해당 행만 계산. 단어장을 유일한 기준으로 두고 옛 용어표 이관은 별도 데이터 작업으로 분리

4. GPT 일괄 번역 도구 → 검수 전용 앱으로 재편

As-Is	6단계 파이프라인(정규화 → 용어 → 번역 → 검수 → 내보내기)이 앱의 뼈대, 실제 팀은 이미 번역된 파일의 검수에만 사용
Challenge	쓰지 않는 화면이 앱의 절반이고 검수 그리드는 겹가지. 2주간 만든 화면을 지우되 데이터와 API는 깨지지 않아야 함
To-Be	화면 4개(게임 · 파일 · 정규화 · 그리드)로 축소하고 그리드를 중심에. 백엔드는 그대로 두고 GPT는 행 단위 번역 · 짧게 줄이기에만

프로젝트 회고

이 프로젝트를 시작할 때의 목표는 단순했다.

“중국어 게임 텍스트 엑셀을 올리면 GPT가 한국어를 채워 주는 도구를 만들자.”

처음 2주는 그 방향으로 갔다. `xlsx`를 정규화해 세그먼트로 나누고, 고유명사를 추출해 용어집을 확정하고, 30행씩 묶어 GPT에 보내고, 길이를 검사해 내보내는 6단계 파이프라인을 만들었다.

자리표시자와 태그가 원문과 다르면 재시도하는 검증도 붙였다.

하지만 실제 팀이 쓰기 시작하자 쓰임새가 달랐다. 번역가가 이미 초벌을 끝낸 5만 행짜리 파일이 들어왔고, 팀이 원한 것은 “번역”이 아니라 여러 명이 동시에 보면서 고치고, 누가 어디까지 봤는지 알 수 있는 검수 화면이었다. 엑셀 복사본을 나눠 보내고 메일로 회수해 합치던 일이 병목이었다.

그래서 6단계 파이프라인을 걷어내고 화면을 네 개로 줄였다. 게임 목록, 파일 올리기, 열 역할 지정, 그리고 엑셀식 검수 그리드. 2주 동안 만든 화면 절반을 지우는 결정이었지만, 백엔드는 그대로 두고 GPT는 행 단위 번역과 짧게 줄이기에만 남겨 위험을 줄였다.

가장 어려웠던 것은 5만 행을 여러 명이 동시에 고치는 상황을 데이터베이스가 감당하게 만드는 일이었다. 첫 목록 API는 쪽마다 프로젝트 전체 리비전을 메모리에 올려 쪽당 2.3초, 620MB를 썼고 1GB 서버에서는 화면이 비어 있었다. 현재 번역을 트리거로 유지하는 `latest_ko` 열을 두고 시트 안에서 DB 페이지징으로 바꾸자 쪽 이동이 즉시가 됐다.

동시 편집도 문제였다. 두 사람이 같은 행의 Fix와 체크를 고치면 나중 저장이 앞 저장을 되돌렸다. 원인은 JPA `save()`가 엔티티의 모든 열을 UPDATE하는 것이었고, 요청이 소유한 열만 쓰는 부분 업데이트로 바꾼 뒤 커밋 후 SSE로 바뀐 열만 전파하게 했다.

검수자들이 가장 반긴 기능은 화려한 것이 아니었다. 같은 원문이 게임 전체에서 313번 나오는데 한 곳만 고쳐지는 문제를 “저장하면 같은 원문 행을 함께 고친다”로 풀었고, 단어장 규칙은 게임 전체 Fix에 일괄 적용되게 했다. 이 둘이 표기 통일에 드는 시간을 가장 많이 줄였다.

깨달은 점

첫 번째로 배운 것은 처음 세운 문제 정의를 버릴 줄 알아야 한다는 점이다.

“GPT 번역 도구”라는 정의에 맞춰 만든 화면이 많았지만, 실제 팀의 시간이 가장 많이 새는 곳은 검수와 합치기였다. 쓰이지 않는 기능을 지우고 그리드를 중심에 두자 도구가 비로소 매일 열리기 시작했다. 만든 것에 애착을 갖기보다 어디서 시간이 새는지 다시 재는 편이 훨씬 빨랐다.

두 번째는 규모가 설계를 바꾼다는 점이다.

6천 행에서 잘 돌던 코드가 5만 행에서는 화면을 비웠다. 목록은 DB에서 자르고, 파생값은 트리거로 캐시하고, 요약은 집계 쿼리로 따로 구하는 세 원칙을 세운 뒤에야 쪽 이동이 즉시가 됐다. 용어 불일치 다시 계산이 10분 걸리는 문제는 버튼을 막아 두고 저장 시점 증분 계산으로 바꾸는 중이다.

세 번째는 동시 편집에서 “무엇을 쓰는가”가 “무엇을 읽는가”만큼 중요하다는 점이다.

한 요청이 자기 열만 쓰게 하는 규칙 하나가 잠금이나 충돌 대화상자 없이도 서로의 수정을 지키게 했다. 이를 위해 `detached` 엔티티를 `save()`하지 않는 관례를 코드 리뷰 기준으로 삼았다.

마지막으로, 운영 데이터를 로컬에서 재현할 수 있어야 안심하고 배포할 수 있다는 점이다.

운영 DB와 올린 파일을 그대로 가져오는 스크립트로 마이그레이션과 성능을 실제 크기의 데이터로 확인한 뒤 배포했다. 작은 도구라도 운영 절차가 곧 제품의 일부라는 것을 이번 프로젝트에서 배웠다.

PROJECT.3

ThePlayPlus Help Center

03



프로젝트 기간	2026.8.28 ~ 2026.09.09
프로젝트 현황	개발 완료
팀 구성	개인 프로젝트
역할	풀스택 개발
도메인	https://www.theplayplus.com/contact
Git (관리자)	github.com/Hong9915/theplayplus-admin
Git (접수 폼)	github.com/Hong9915/theplayplus-contact

Why — 기존 THEPLAYPLUS(게임 퍼블리셔 대행사) CS 문제점

기존 모든 Consumer service를 이메일을 통해 답변

☐ ★ 나, 조광현 9	[여신키우기: 방치형 RPG] 게임 콘텐츠 관련 문의 드립니다 - 네 확인하였습니다 감사합니다	오후 4:21
☐ ★ 나, BLUE 18	[여신키우기: 방치형 RPG] 게임 콘텐츠 관련 문의 드립니다 - 네 감사합니다. 수고하세요. 2026년 9월 10일 (목) 오후 3:24, 강범준 ... image.png	오후 3:44

문제해결	반복되는 행동	게임사와의 의사소통
문의 1건 해결에 매일 최대 15회 왕복 이메일 스레드 구조상 문의 이력과 처리 상태를 한눈에 파악하기 어려워, 해결까지 시간이 오래걸림 How + Result : 문의 관리 시스템도입 -> 하루 2회 확인	같은 유형의 문의에 매번 수동으로 답변 쿠폰, 결제 등 유사 문의가 반복되지만 매번 직접 작성하거나 복사·붙여넣기로 대응 How + Result : FAQ·자동응답·템플릿 구축	유저<->대행사<->게임사, 3단계를 거치는 구조 게임사 확인이 필요한 문의는 메일로 다시 전달,회신을 기다려야 해 응답이 지연되고 정보가 누락됨 How + Result : 게임사와 공유되는 문의 채널/대시보드 구축

CS 인박스

접수 폼으로 들어온 문의를 한 화면에서 분류·자동 응대·답변·추적하는 관리자 워크플로우

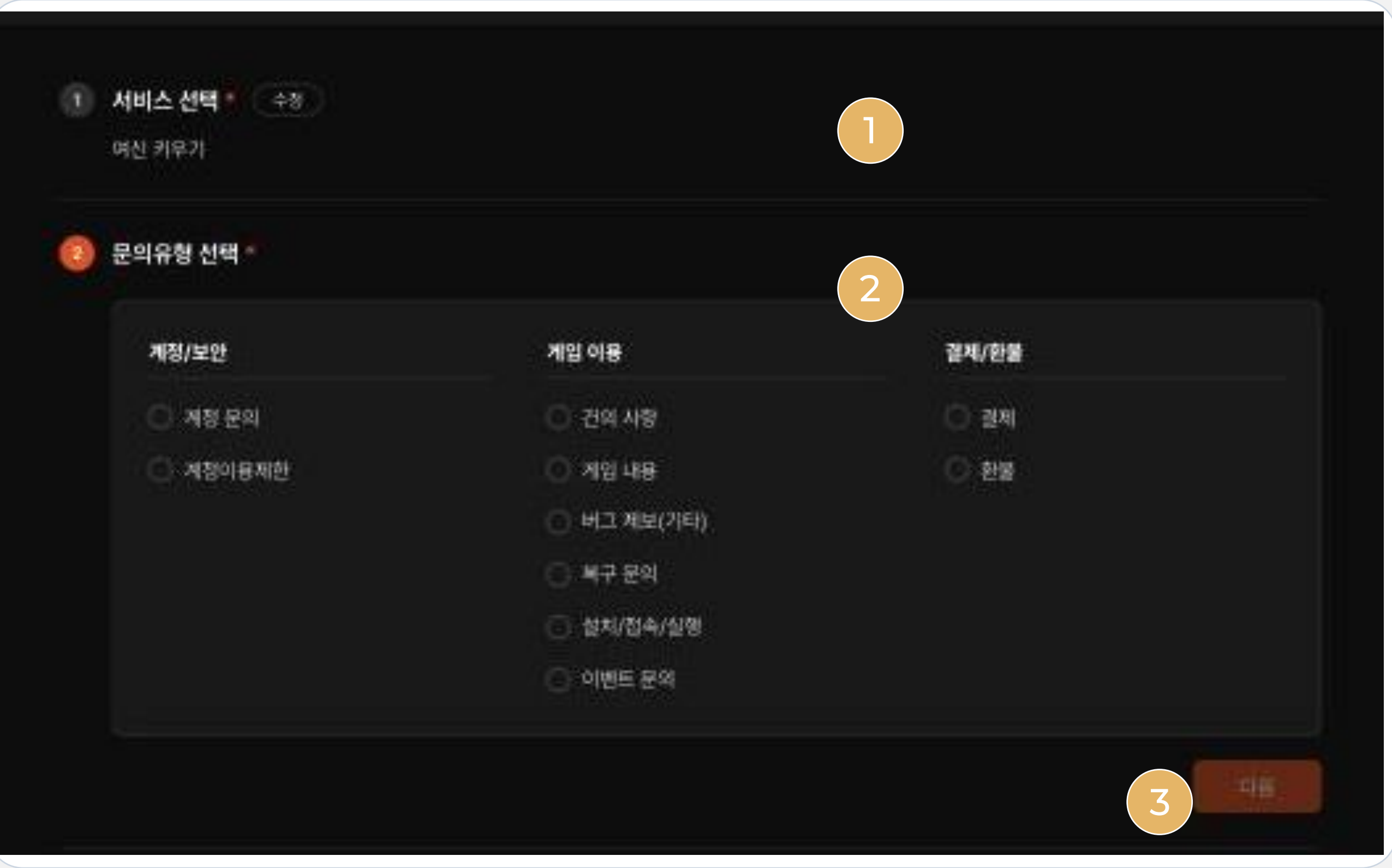


주요 화면 · 기능

- IN-01 게임 레일 · 접수 건수 배지 · 서비스 문의함
- IN-02 문의함 보기 (상태 · 유형 · 우선순위 · 3일 미처리 · 회신 도착)
- IN-03 문의 목록 · 검색 · 정렬 · 일괄 상태 변경
- IN-04 대화 타임라인 (본문 · 답변 · 회신 · 메모 · 같은 계정 이어보기)
- IN-05 답변 / 메모 작성란 · 초안 자동 저장
- IN-06 상세 패널 · 계정 이력

- AD-01 게임 관리 (기본 유형 · 템플릿 자동 복사)
- AD-02 답변 템플릿 · 자동 발송 설정
- AD-03 운영 자료 연결 (구글 시트 · 문서)
- AD-04 AI 답변 추천 · 대화 번역
- AD-05 Slack 긴급 알림 · 회신 자동 동기화 (pg_cron)

기능 소개 01 — 접수 폼 · 게임별 문의 유형



1 서비스 선택

관리자에서 등록한 게임 목록을 그대로 노출.
게임이 아니면 "서비스 문의(제휴·기타)"로 분기

2 문의 유형 (게임별 DB 관리)

계정/보안 · 게임 이용 · 결제/환불 3그룹 10유형.
하드코딩 없이 inquiry_groups / inquiry_types에서 읽고
새 게임을 만들면 기본 유형·자동 답변 템플릿이 복사됨

3 접수 즉시 DB 트리거

유형의 default_priority로 우선순위 지정
(결제·환불·복구 = 긴급, 건의 = 낮음)
자동 답변 예정 시각(5~30분 뒤)을 기록

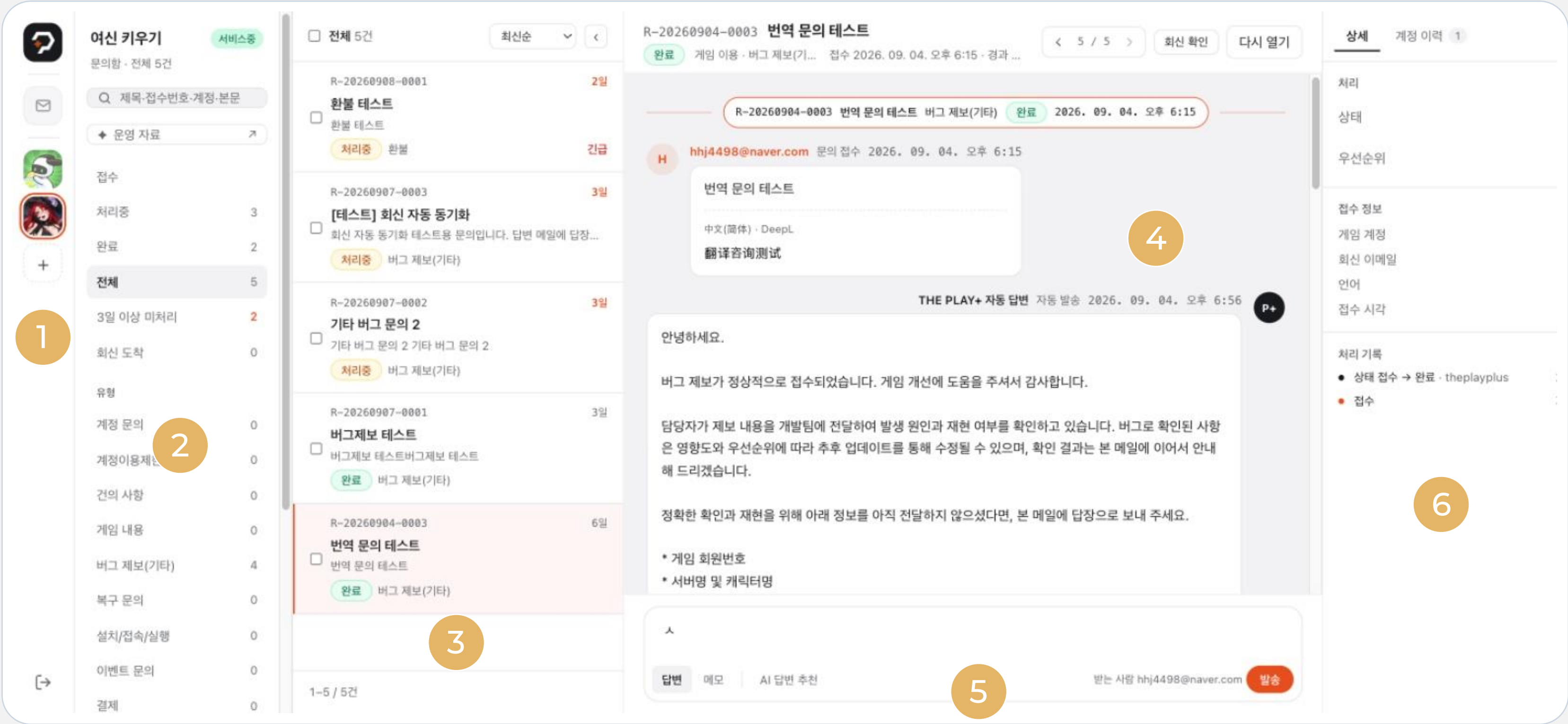
4 긴급 건 Slack 알림

Database Webhook → /api/notify/inquiry →
긴급 게임 문의 · 서비스 문의만 Slack으로



게임 타일은 관리자에서 등록한 게임이
그대로 노출된다 (games 테이블 · 로고 Storage)

기능 소개 02 — 인박스 한 화면 (게임 레일 · 문의함 보기 · 목록 · 대화 · 상세)



1 게임 레일

게임별 문의함 전환 · 접수(new) 건수 배지
맨 위 타일은 서비스 문의(game_id null)

2 문의함 보기

상태 · 유형 · 우선순위 · 3일 미처리 · 회신 도착
건수는 inquiry_facet_counts RPC 한 번으로

3 문의 목록

URL 쿼리로 필터·정렬·검색·페이지 관리
체크박스로 여러 건 상태 일괄 변경

4 대화 타임라인

본문 · 답변 · 회신 · 메모를 시간순 한 줄기로
같은 계정의 다른 문의도 구분선 두고 이어보기

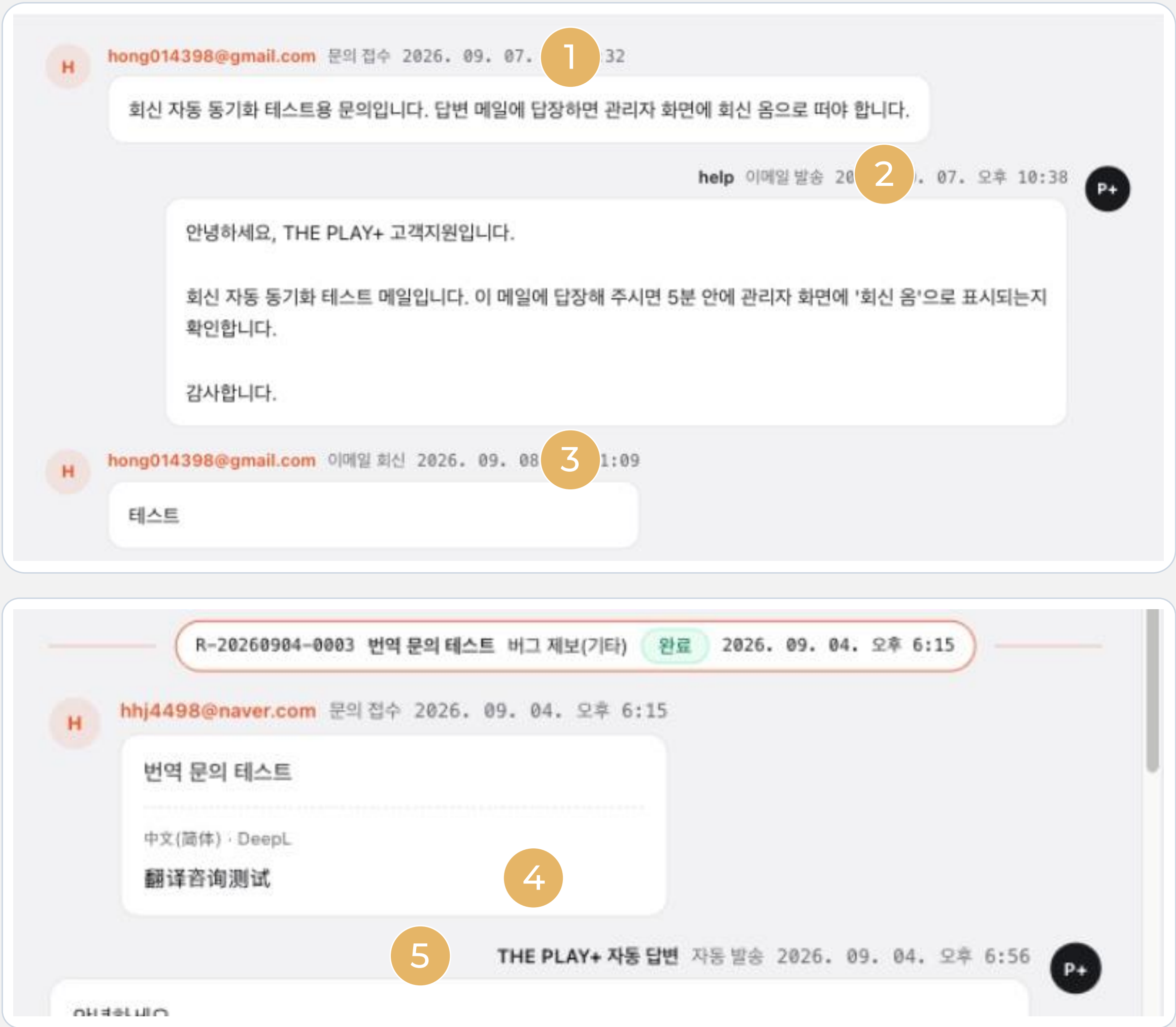
5 답변 / 메모 작성란

Cmd+Enter 발송, 초안은 2초 뒤 자동 저장

6 상세 · 계정 이력

상태·우선순위 변경, 처리 기록, 같은 계정 과거 문의

기능 소개 03 — 자동 답변 · Gmail 스레드 답변 · 회신 자동 동기화 · 번역



1 문의 접수

접수 폼에서 들어온 본문 · 첨부가 첫 말풍선

2 Gmail API 답변 발송

브랜드 HTML + 텍스트 multipart로 발송,
후속 답변은 같은 스레드에 묶임. 발송 시 상태 → 처리중

3 회신 자동 동기화

pg_cron 5분 주기로 메일함의 새 메일을 스레드 id로 매칭.
"회신 옴" 배지 · 회신 도착 보기, 열면 읽음 처리

4 대화 번역 (DeepL)

말풍선 우클릭 → 한국어 / 중국어(간체) 번역.
결과는 jsonb에 언어별 저장돼 다시 열어도 유지

5 유형별 자동 답변

접수 5~30분 뒤 템플릿을 자동 발송 (타임라인에 "자동 발송").
상태는 접수로 유지, 관리자가 먼저 답하면 건너뛴

기능 소개 04 — AI 답변 추천 · 답변 템플릿 · 운영 자료 연결

답변을 입력하세요. 꺾 Enter로 발송합니다.

답변

메모

1

쓰는 중

중단

받는 사람 hong014398@gmail.com

발송

회신 잘 받았습니다. 관리자 화면에 '회신 옴'으로 정상 표시된 것을 확인했으며, 테스트는 여기서 마무리하겠습니다.

감사합니다.

답변

메모

AI 답변 추천

받는 사람 hong014398@gmail.com

발송

초안을 다듬었습니다. 검토 뒤 발송하세요. 되돌리기

등록된 템플릿

자동 발송을 켜면 그 유형의 새 문의에 이 템플릿이 5분~20분 뒤 자동으로 발송됩니다. 같은 유형에는 하나만 켤 수 있습니다.

결제 문의 접수 안내

자동 발송 중

수정

자동 발송 끄기

결제

안녕하세요.

결제 관련 문의가 정상적으로 접수되었습니다.

담당자가 스토어(Google Play / App Store / ONE store)의 결제 기록과 게임 내 상품 지급 내역을 대조하여 확인하고 있습니다. 확인이 완료되는 대로 본 메일에 이어서 결과를 안내해 드리겠습니다.

확인에는 영업일 기준 1~3일이 소요될 수 있는 점 양해 부탁드립니다.

빠른 확인을 위해 아래 정보를 아직 전달하지 않으셨다면, 본 메일에 답장으로 보내 주세요.

- 1

AI 답변 추천 (스트리밍)

버튼을 누르면 작성란에 바로 써짐 (주황 점선 · 중단 가능).
근거: 작성 중 초안 · 대화 이력 · 유형 템플릿 ·
pgvector 유사 과거 답변 · 운영 자료 (OpenAI gpt-5-mini)
- 2

다듬기 · 되돌리기

초안이 있으면 새로 쓰지 않고 초안의 결정을 유지해 다듬음.
"되돌리기"로 누르기 전 글 복구, 편집하면 사라짐
- 3

답변 템플릿 · 자동 발송

게임 · 유형별 템플릿, "자동 발송" 켜면 새 문의에 자동 응대.
같은 유형에 하나만 켜짐 (다른 걸 켜면 이전 것은 꺼짐)
- 4

운영 자료 연결

구글 시트 · 문서 URL을 서비스 계정으로 읽어
AI 추천의 근거로 사용 (합계 30만 자까지)

여신 키우기 운영 자료

여기에 연결한 구글 스프레드시트·문서는 문의함의 "AI 답변 추천"이 답변 근거로 읽습니다. 각 자료를 theplayplus-sheet-assistant@long-leaf-424707-h0.iam.gserviceaccount.com에 편집자로 공유한 뒤 추가하세요.

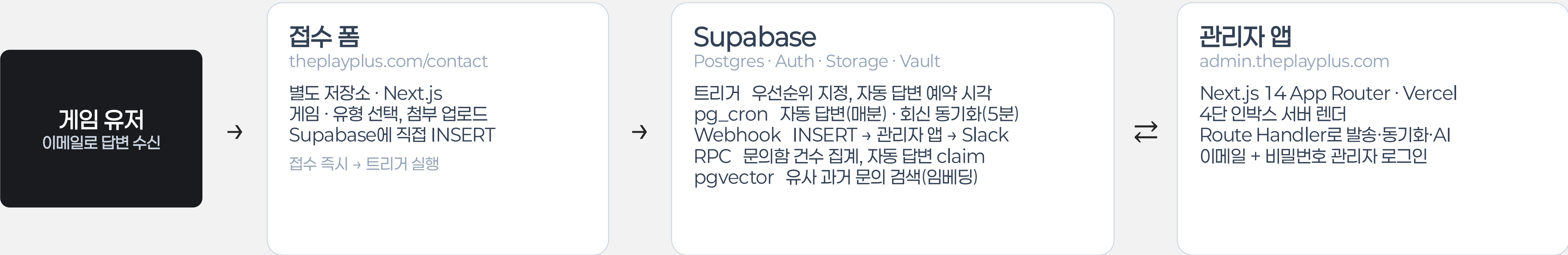
연결된 자료

■ 운영 시트

■ VIP_보상_통합관리

+ 자료 추가

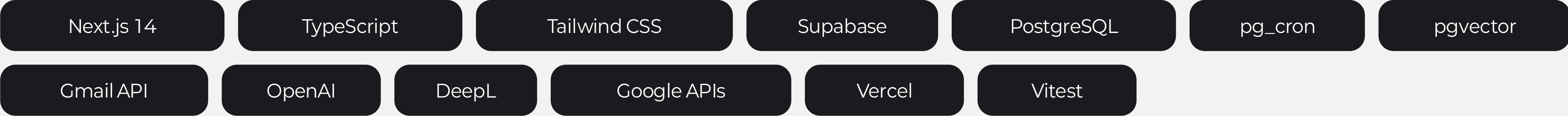
접수 폼과 관리자 앱이 하나의 Supabase 프로젝트를 공유하고, 주기 작업은 DB 안에서(pg_cron) 돈다



관리자 앱이 부르는 외부 서비스



기술 스택



인박스 · 자동화 · AI 보조 구현

4단 인박스 화면

게임 레일 · 보기 · 목록 · 대화 · 상세를 한 페이지 서버 렌더
필터 · 정렬 · 검색은 URL 쿼리, 건수는 facet RPC 한 번으로 집계
[Github : github.com/Hong9915/theplayplus-admin](https://github.com/Hong9915/theplayplus-admin)

GMAIL 답변 발송 · 회신 자동 동기화

브랜드 HTML multipart 발송, 후속 답변은 같은 스레드로 묶음
pg_cron 5분 주기로 새 메일을 스레드 id로 매칭해 타임라인 반영

유형별 자동 답변 (5~30분 지연)

트리거가 예정 시각을 찍고 pg_cron이 매분 claim RPC로 발송
호출이 겹쳐도 1회, 관리자가 먼저 답하면 건너뛴

AI 답변 추천 (스트리밍)

초안 · 대화 이력 · 템플릿 · 유사 과거 답변 · 운영 자료를 근거로
pgvector 유사도 검색 + OpenAI, 작성란에 바로 써지고 근거 표시

SLACK 긴급 알림 · DEEPL 번역

DB Webhook → 긴급 · 서비스 문의만 골라 Slack 발송
말풍선 우클릭 번역, 결과는 jsonb에 언어별 저장

운영 자료 연결 · 접수 폼

구글 시트 · 문서를 서비스 계정으로 읽어 AI 근거로 사용
[Github : github.com/Hong9915/theplayplus-contact](https://github.com/Hong9915/theplayplus-contact)

Why — 각 기술 선택의 근거

Q 왜 별도 API 서버 없이 Supabase 하나로?

접수 폼과 같은 Supabase 프로젝트를 공유해 데이터를 바로 읽고, Vercel 서버리스에는 상시 프로세스가 없음.
우선순위·자동 답변 예약은 DB 트리거, 주기 작업은 pg_cron, 알림은 Database Webhook으로 DB 안에서 해결

Q 왜 자동 답변을 즉시 웹훅이 아니라 pg_cron + claim RPC로?

접수 직후 답변은 기계적인 인상을 주고, 웹훅 재시도·동시 호출이 같은 문의에 두 번 보낼 위험이 있음.
5~30분 랜덤 지연 후 RPC가 예정을 지우면서 행을 돌려줘 겹쳐도 1회 발송, 관리자가 먼저 답했으면 건너뛸

Q 왜 새 채팅 채널이 아니라 Gmail API 스레드?

유저는 이메일로 답을 받는 데 익숙하고 게임사 전달도 메일이라 채널을 바꾸면 이력이 두 곳으로 갈라짐.
답변·회신을 같은 스레드에 묶고 messages.list로 주기 수집해 이메일 UX는 그대로, 관리자 화면은 채팅처럼

도입 전후 비교 및 최종 결과

업무	도입 전 (이메일)	도입 후 (cs 인박스)	구현	효과
문의 확인	Gmail 수신함, 하루 최대 15회 왕복	상태 · 우선순위 · 미처리 필터로 하루 2회 확인	URL 쿼리 필터 + facet RPC	확인 횟수 15 → 2
첫 응답	담당자가 볼 때까지 대기	접수 5~30분 안에 유형별 자동 답변	트리거 + pg_cron + claim RPC	첫 응답 30분 이내
긴급 건 인지	메일을 열어봐야 알 수 있음	결제 · 환불 · 복구 문의는 Slack 즉시 알림	DB Webhook → Slack	누락 방지
반복 답변	이전 메일 복사 · 붙여넣기	템플릿 · 유사 답변 · 운영 자료 근거 AI 초안	OpenAI + pgvector	작성 시간 단축
회신 추적	스레드를 직접 찾아 확인	5분마다 자동 동기화, "회신 옴" 배지	Gmail messages.list + sync state	놓치는 회신 없음

15 → 2

하루 문의 확인 횟수 | 이메일 왕복 → 인박스

≤ 30분

첫 자동 응답 | 5~30분 랜덤 지연

5분

회신 자동 반영 주기 | pg_cron + Gmail API

1. 자동 답변 중복 발송 (웹훅 → pg_cron + claim RPC)

As-Is	Supabase 웹훅이 INSERT 즉시 발송 API를 호출. 접수 직후 답변이 나가고 재시도 시 중복 발송
Challenge	웹훅 재시도·동시 호출에도 한 번만 보내야 하고, 사람이 답한 것처럼 자연스러운 시점이 필요
To-Be	트리거가 5~30분 뒤 예정 시각을 기록하고 pg_cron이 매분 claim RPC로 예정을 지우며 가져와 발송. 겹쳐도 1회, 관리자 선택변 시 건너뛴

2. AI 추천이 작성 중 초안을 무시 (프롬프트 구조)

As-Is	작성란의 초안을 근거 자료 뒤에 한 구간으로 덧붙여 모델에 전달
Challenge	gpt-5-mini가 템플릿·이력으로 새 답변을 쓰고 초안의 결정을 뒤집음 (2026-09-08 실측)
To-Be	초안이 있으면 "이번 작업: 초안 다듬기"로 시작해 초안을 먼저 보이고 자료는 참고 정보로, 끝에 한 번 더 상기. 결과는 "되돌리기"로 복구

3. 회신 확인 수동 → 자동 동기화

As-Is	관리자가 "회신 확인"을 눌러야 그 문의의 Gmail 스레드를 조회
Challenge	문의마다 스레드를 조회하면 API 호출이 많고, 누르지 않은 문의의 회신을 놓침
To-Be	pg_cron 5분마다 메일함별 "마지막 확인 이후" 메일을 messages.list 한 번으로 받아 스레드 id로 매칭. 10분 겹쳐 훑고 message id 유니크로 중복 제거

4. 운영 시트 어시스턴트 채팅 → 자료만 유지

As-Is	구글 시트·문서를 읽어 답하고 시트 수정까지 제안하는 채팅 화면을 구현
Challenge	답변 품질이 운영에 쓸 수준에 못 미쳤고, 오답이 시트 수정 제안으로 이어질 위험
To-Be	채팅은 플래그로 기본 비활성화(API 404), 자료 등록 화면만 남겨 AI 답변 추천의 근거로 계속 사용. 수정은 제안 → 관리자 승인 구조 유지

프로젝트 회고

이 프로젝트를 시작할 때의 목표는 단순했다.

“이메일로만 받던 고객 문의를 하나의 화면에서 처리할 수 있게 만들자.”

하지만 실제로 개발을 시작하니 단순히 문의를 접수하는 페이지를 만드는 것만으로는 부족했다. 관리자가 문의에 빠르고 편리하게 답변할 수 있어야 했고, 사용자의 이전 문의와 처리 기록도 추적할 수 있어야 했다. 긴급한 문의와 일반 문의를 구분하는 기능도 필요했다.

한국인 관리자뿐만 아니라 중국인 관리자도 사용하는 페이지였기 때문에 번역 기능도 추가해야 했다. 답변을 작성할 때는 기존 LLM에 RAG 방식으로 관련 자료를 전달한 뒤 결과를 복사해 사용할 수도 있었지만, 해당 사용자와 나눈 대화 내용까지 반영한 답변이 필요했다. 이를 위해 CS 문의 페이지 안에 ‘AI 답변 추천’ 기능을 추가했다.

이 과정에서 진짜 어려웠던 것은 화면을 구현하는 일이 아니었다. **문의 접수·답변 작성·자동 발송·사용자 회신 수집이 서로 꼬이지 않도록 하나의 시간 흐름을 설계하는** 일이 가장 어려웠다. 초기에는 문의 접수 웹훅에서 자동 답변을 즉시 발송했다. 응답 속도는 빨랐지만, 웹훅이 재시도되면 같은 문의에 답변이 중복 발송되는 문제가 발생했다. 문의가 접수된 지 1초 만에 답변이 전송되는 모습도 사용자에게 지나치게 기계적인 인상을 줄 수 있었다.

결국 자동 답변의 발송 시점을 데이터베이스 트리거가 결정하고, pg_cron이 claim RPC를 통해 발송 대상을 가져가도록 구조를 변경했다. 이를 통해 **누가, 언제, 어떤 메시지를 발송하는지와 동일한 답변이 한 번만 발송된다는 사실을 데이터베이스 수준에서 보장할 수 있었다.**

사용자의 이메일 회신을 동기화하는 방식도 개선했다. 처음에는 문의마다 Gmail을 개별 조회했지만, 문의가 늘어날수록 요청 횟수와 처리 시간이 함께 증가했다. 이를 메일함 단위의 증분 조회 방식으로 변경해 새로 들어온 회신만 효율적으로 수집하도록 만들었다.

깨달은 점

가장 인상 깊었던 것은 **SI가 관리자가 작성한 초안을 무시하던 문제였다.**

처음에는 모델의 성능 문제라고 생각했지만, 실제 원인은 프롬프트의 순서였다. 관리자가 작성한 초안을 참고 자료 뒤에 배치하면 모델은 초안을 수정하기보다 자료를 바탕으로 새로운 답변을 작성했다. 반대로 초안을 프롬프트의 가장 앞에 두고 ‘이번 작업에서 우선적으로 따라야 할 결정’으로 명시하자, 모델이 관리자의 의도와 결정을 훨씬 잘 유지했다.

이를 통해 같은 모델과 같은 자료를 사용하더라도 **어떤 정보를 먼저 보여주고, 각 정보에 어떤 역할을 부여하느냐에 따라 결과가 크게 달라질 수 있다**는 점을 배웠다.

두 번째로 배운 것은 **만든 기능을 끝 줄 아는 것도 중요한 판단**이라는 점이다.

운영 시트 어시스턴트 채팅 기능은 구현을 완료했지만, 실제 운영자가 신뢰하고 사용할 수 있을 만큼 답변 품질이 안정적이지 않았다. 이에 기능 플래그를 통해 기본적으로 비활성화하고, 연결해 두었던 운영 자료만 유지했다. 해당 자료는 이후 AI 답변 추천 기능의 근거 데이터로 활용할 수 있었다.

이 경험을 통해 기능의 성공 기준을 단순히 ‘정상적으로 동작하는가’가 아니라, **‘실무자가 믿고 사용할 수 있는가’**로 바라보게 되었다. 기술적으로 구현이 끝났더라도 운영 품질이 기준에 미치지 못한다면 공개하지 않는 것이 더 나은 선택일 수 있다.

세 번째로 배운 것은 **작은 시스템에서도 운영 설계가 사실상 전부**라는 점이다.

자동 답변 발송에 실패하면 5분 뒤 다시 발송하도록 예약하고, Gmail 조회에 실패하면 마지막 확인 시각을 갱신하지 않아 다음 실행에서 같은 구간을 다시 확인하도록 설계했다. 단순히 정상적인 상황에서 기능이 작동하게 만드는 것보다, 오류가 발생했을 때 데이터가 누락되거나 중복되지 않고 다시 정상 상태로 돌아올 수 있도록 만드는 것이 더 중요했다.

이번 프로젝트를 통해 기능을 구현하기 전에 **실패했을 때의 데이터 흐름과 복구 경로를 먼저 설계하는 습관**을 갖게 되었다. 결국 안정적인 CS 시스템을 만드는 핵심은 많은 기능을 추가하는 것이 아니라, 문의가 접수되고 답변과 회신이 오가는 모든 과정에서 데이터와 시간이 어긋나지 않도록 설계하는 일이었다.

PROJECT.4

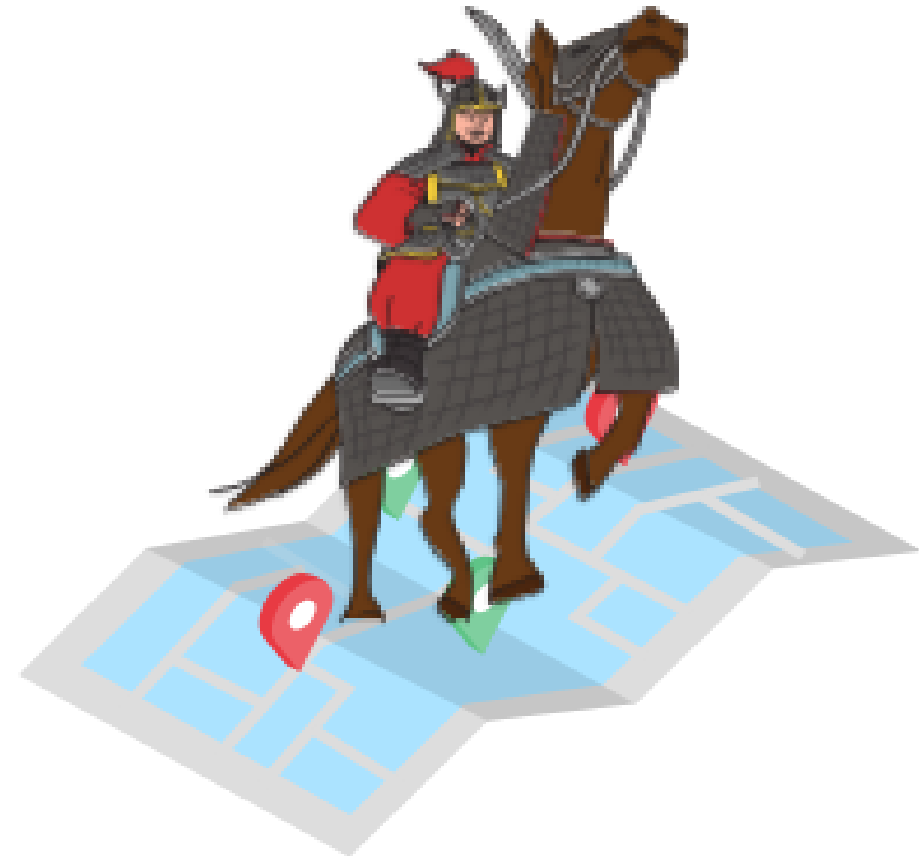
H.AI

Gen AI 기반 학생 참여형 역사 교과서

04



프로젝트 기간	2025.11.07 ~ 2026.01.23
프로젝트 현황	개발 완료
팀 구성	프론트 3명 백엔드 4명
역할	백엔드 개발 및 클라우드 인프라 설계
수상	대상
기여도	30%
Git	https://github.com/H-AI-LG
시연영상	https://youtu.be/c3KzTTY3fmY



지도 기반 시각화

시대에 따라 국경이 변하는 지도

지도 위에서 움직이는 캐릭터

이해도 향상



AI 인물 대화

LLM 기반 역사 인물과 채팅

역사 인물과 실시간 통화

교과서에서 인물 불러오기

창의력 향상

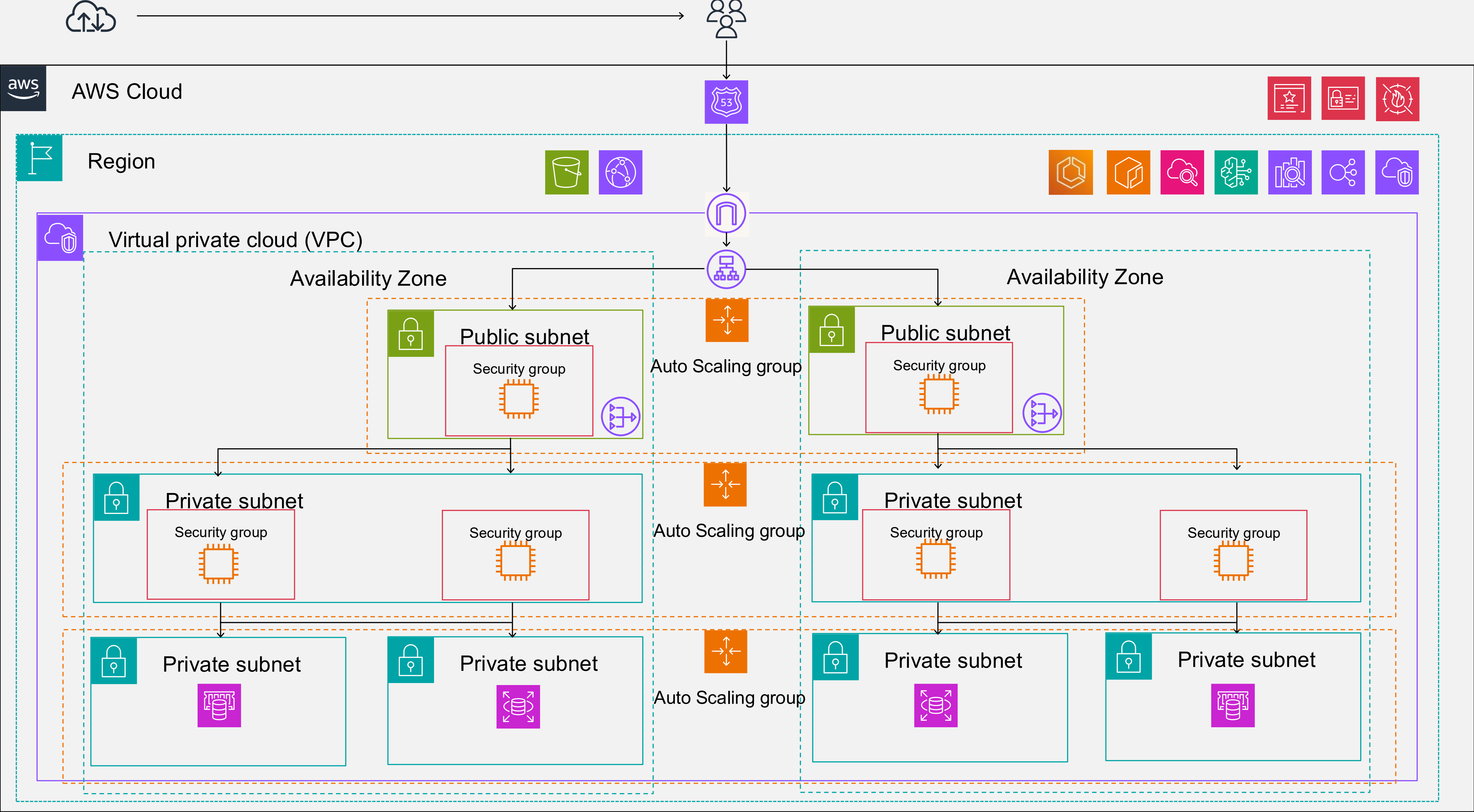


교과서 내 AI Agent

실제 교과서 RAG 기반 답변

Tool Calling을 이용한 조작

사용감 개선



지도 기반 시각화



Leaflet
geojson

AI 인물 대화



AWS Bedrock



Typecast



Claude

실시간 온라인 토론



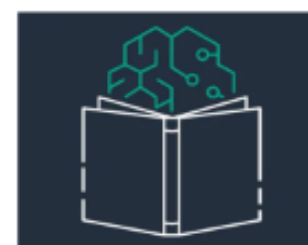
AWS Bedrock

web socket
Stomp 

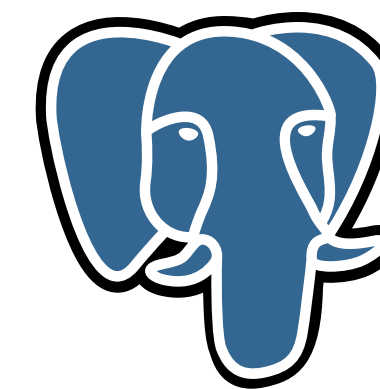
교과서 기반 RAG 챗봇



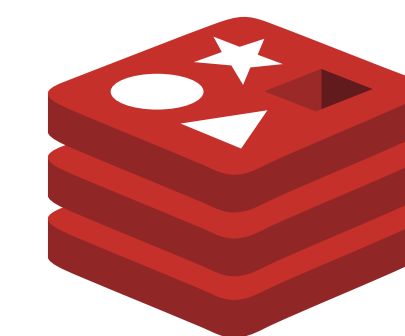
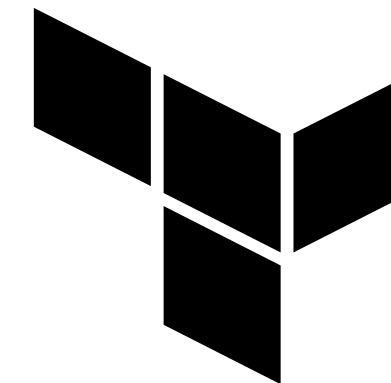
AWS Bedrock



AWS Bedrock Knowledge base



dj



AI 기능 구현 및 인프라

AI 역사 인물 대화

Bedrock 스트리밍 + SSE 실시간 응답

Github : <https://github.com/H-AI-LG/py-backend/blob/main/apps/prompt/views.py>

교과서 RAG 챗봇

Knowledge Base + Tool Calling으로 의도 자동 분기

교과서 PDF 청킹 시 페이지 번호를 메타데이터로 첨부하여 S3 Vector 저장

실시간 토론

Redis로 대화 맥락 유지 + 결과 요약 생성

Github : <https://github.com/H-AI-LG/py-backend/blob/main/apps/debate/views.py>

AWS ECS FARGATE 3-TIER

Presentation tier + Application tier + Data tier 으로 나눠서 각각의 계층 분리 후 설계

Github : <https://github.com/H-AI-LG/terraform>

CI/CD 파이프 라인 구축

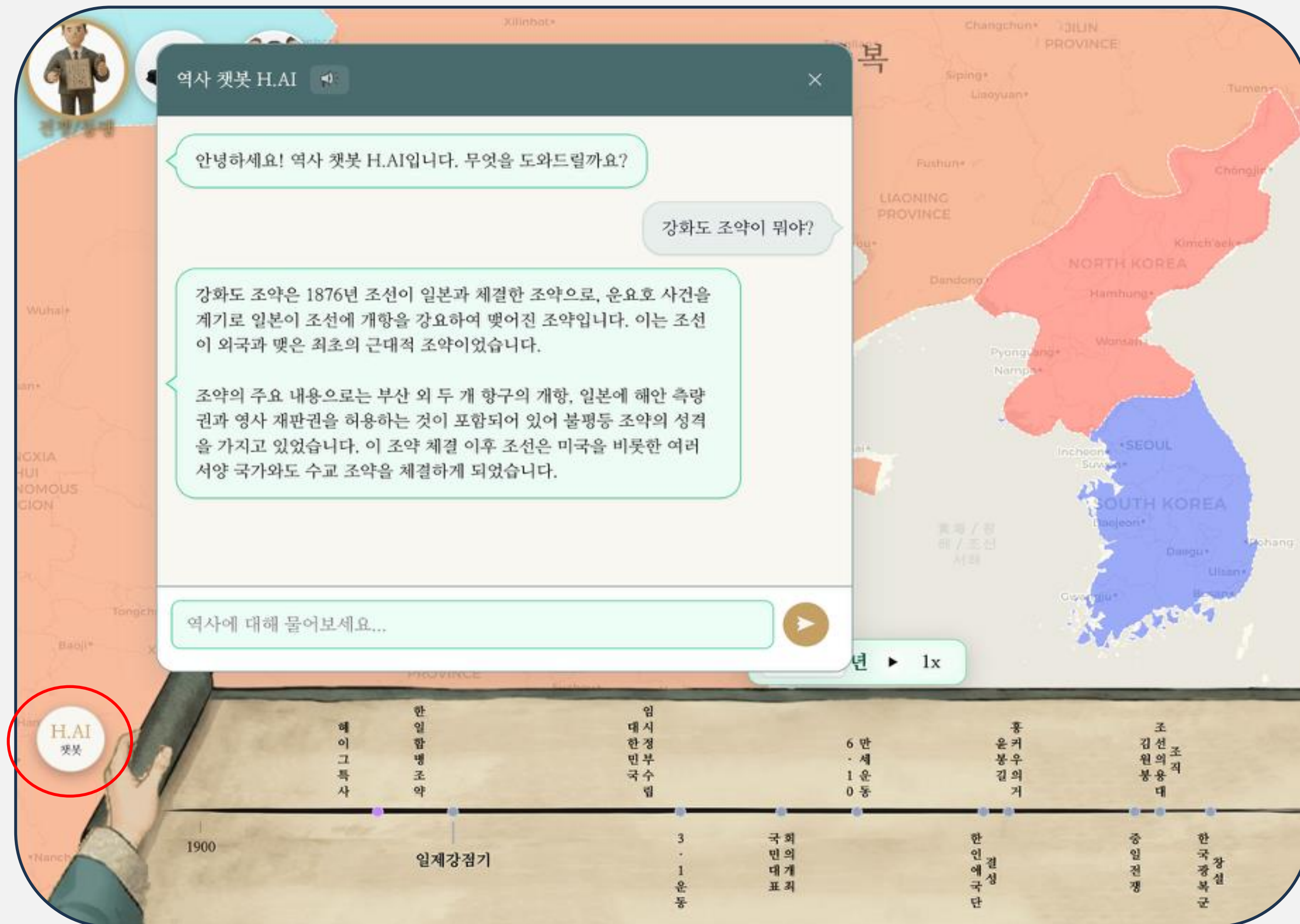
Github Actions -> ngrok -> Jenkins -> ECR -> ECS Fargate

CLAUDE SONNET 채택

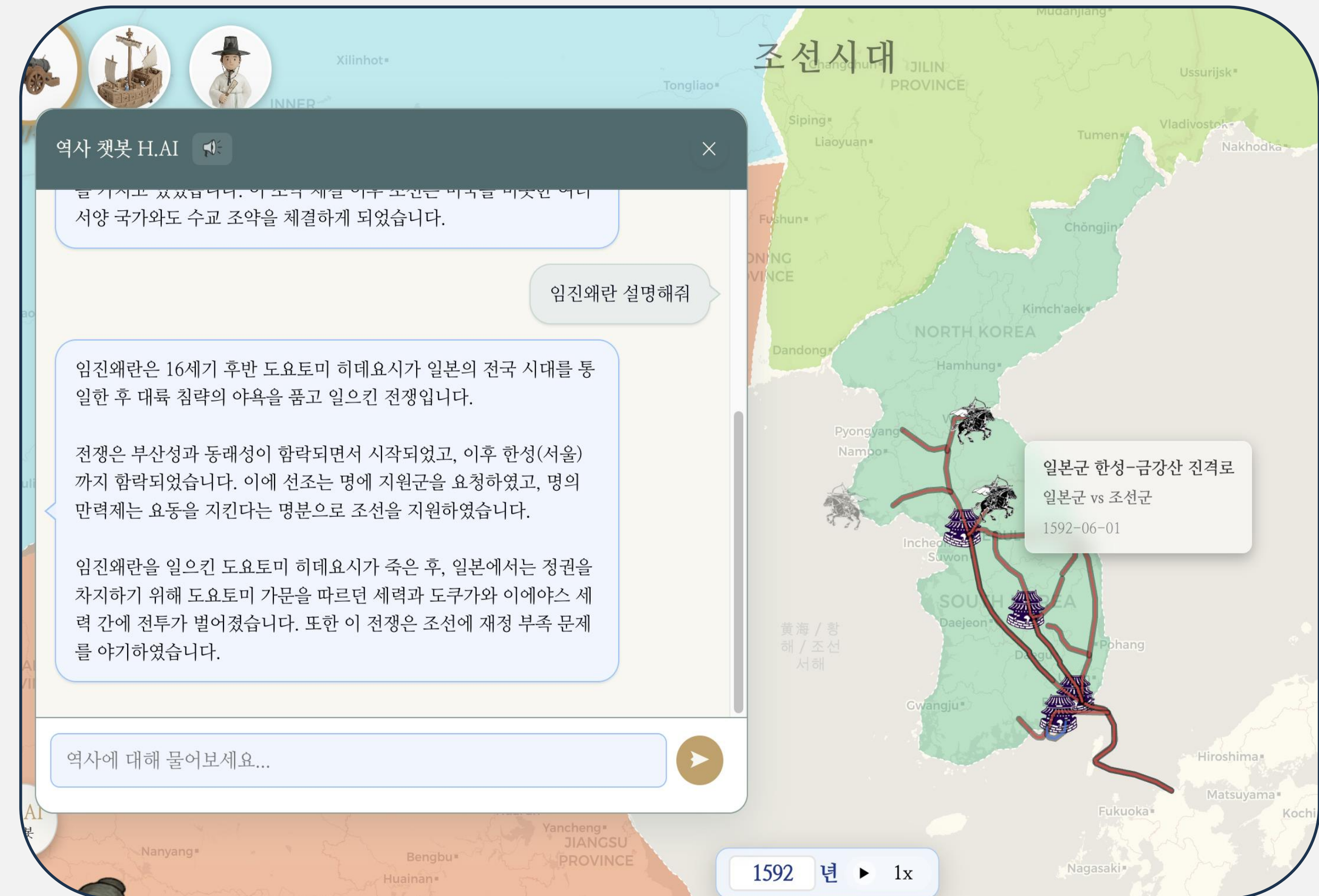
테스트 셋 : 한국사검정능력 시험 50문제 + 한국어능력시험 50문제

최종 점수 = 성능(정답비율)40% + 속도40% + 비용효율 20%

1. 교과서 RAG 기반 답변



2. Tool Calling 기능



1. 이중화 (Active-Standby)

As-Is	단일 ECS 태스크로 Spring Boot, Django 서버 운영. 장애 발생 시 서비스 전체 중단 위험
Challenge	고가용성 확보 필요. 트래픽 급증이나 인스턴스 장애 시 무중단 서비스 보장이 어려움
To-Be	ECS 서비스의 desired count 설정 및 Multi-AZ 배포로 Active-Standby 구성. 하나의 태스크 장애 시 자동으로 다른 인스턴스가 트래픽 수용

2. EKS vs ECS

As-Is	컨테이너 오케스트레이션 도구 선정 필요. EKS(Kubernetes)와 ECS 중 검토
Challenge	EKS는 확장성·표준성이 우수하나 학습 비용이 높고 프로젝트 규모 대비 오버엔지니어링 우려
To-Be	ECS 채택. Kubernetes의 고급 기능(서비스메시, CRD 등)이 불필요했고, ECS만으로 서비스 운영·확장이 충분. ECR과의 연동도 네이티브하게 지원됨

3. SCG (Spring Cloud Gateway) & Route 53 설계 이유

As-Is	Spring Boot, Django 각각에 인증 로직이 중복 구현되어 있었음
Challenge	인증 로직 이중화로 유지보수 비용 증가, 변경 시 두 서버 모두 수정 필요
To-Be	SCG를 단일 진입점으로 두어 인증/인가를 중앙 집중화. Route 53으로 도메인을 실제 엔드포인트에 연결하여 트래픽 라우팅 일원화

4. Knowledge Bases를 통한 Rag vs 직접 구축 RAG

As-Is	AWS Bedrock Knowledge Bases를 활용한 RAG 구성 검토
Challenge	Knowledge Bases는 청킹·임베딩 파이프라인이 추상화되어 있어 커스텀 메타데이터 삽입, 청크 전략 조정 등 세밀한 제어가 어려움
To-Be	교과서 PDF를 직접 파싱하여 LangChain 기반 RAG 직접 구축. page 번호를 metadata에 포함시켜 출처 추적 가능하게 함. S3(Vector Bucket) + OpenSearch로 검색 파이프라인 구성

5. 왜 PostgreSQL(RDS)을 사용했는가?

As-Is	RAG 구축을 위해 벡터 검색 가능한 DB 필요
Challenge	MySQL, MariaDB 등 일반적인 RDB는 벡터 저장·유사도 검색을 지원하지 않음. 별도 벡터 DB를 추가하면 아키텍처 복잡도 증가
To-Be	PostgreSQL의 pgvector 확장 을 활용해 벡터 저장과 유사도 검색을 RDS 단일 인스턴스에서 처리. 관계형 데이터와 벡터 데이터를 함께 관리 가능하여 RAG 파이프라인 구성에 적합

6. Tool Calling을 활용한 AI Agent 구현

As-Is	사용자 입력을 단순 텍스트로 처리하여 LLM이 일반 답변만 생성하는 구조
Challenge	"이순신 인물 보여줘", "임진왜란 전쟁으로 이동해줘" 같은 UI 조작 명령 과 일반 질문을 구분하지 못하고 모두 텍스트 답변으로만 처리됨
To-Be	AWS Bedrock Tool Calling으로 명령 여부 판별 → Django가 어떤 함수를 실행할지 함수명과 파라미터를 프론트로 전달 → 프론트엔드에 미리 정의된 <code>navigate_to_person()</code> / <code>move_to_war()</code> / <code>move_to_page()</code> 함수를 클라이언트에서 실행. 질문인 경우 Knowledge Base RAG 기반 텍스트 답변 생성

7. Page Num을 메타데이터에 넣은 것

As-Is	교과서 텍스트를 청킹하여 벡터 저장 시 텍스트만 청크되어 저장
Challenge	LLM 답변 시 어느 교과서 몇 페이지 내용을 참조했는지 출처 불명확, 학습 맥락 제공 불가
To-Be	각 chunk에 <code>"metadata": { "page": 120 }</code> 형태로 page 번호를 포함. 답변 생성 시 출처 페이지를 함께 제공하여 교육적 신뢰성 향상

8. DB 이중화 & Redis 단일 구성

As-Is	RDS(PostgreSQL)와 ElastiCache(Redis) 모두 단일 인스턴스로 운영
Challenge	RDS는 장애 시 사용자 데이터 유실·서비스 중단 위험. 반면 Redis는 캐시/세션 용도라 장애 시 재생성 가능
To-Be	RDS는 Multi-AZ 동기식 복제 로 이중화 → Primary 장애 시 Standby가 자동 failover, 데이터 유실 0. Redis는 단일 노드 유지 → 세션/캐시 데이터는 유실되어도 재로그인·재조회로 복구 가능하므로 비용 대비 단일로 충분하다고 판단

프로젝트 회고

이 프로젝트를 시작할 때 목표는 단순했다. "AI로 역사 교과서를 더 재미있게 만들자." 하지만 막상 들어가 보니, 진짜 어려운 건 AI 기능을 구현하는 게 아니라 그 기능들이 하나의 서비스로 안정적으로 돌아가게 만드는 일이었다.

처음 아키텍처를 잡을 때는 Spring Boot 하나로 백엔드를 통일하려 했다. 그런데 AWS Bedrock을 연동하려면 Python 생태계가 훨씬 유리했고, 결국 Spring Boot와 Django를 함께 쓰는 구조가 됐다. 두 서버가 공존하면서 인증 로직이 양쪽에 중복으로 구현되는 문제가 생겼다. Spring Boot에서도 인증, Django에서도 인증. 변경이 생기면 두 곳을 모두 수정해야 했다. 멘토님의 피드백을 받고 나서야 Spring Cloud Gateway로 인증을 중앙 집중화하는 구조로 바꿨다. "코드를 잘 짜는 것"과 "구조를 잘 짜는 것"은 다른 문제라는 걸 그때 처음으로 실감했다.

컨테이너 오케스트레이션 선택도 그랬다. EKS가 더 표준적이고 확장성이 좋다는 건 알고 있었다. 하지만 이 프로젝트에서 Kubernetes의 고급 기능이 필요한 순간은 한 번도 오지 않았다. ECS만으로 운영과 확장이 충분했고, 오히려 EKS를 선택했다면 운영 복잡도만 올라갔을 것이다. 기술 선택은 더 좋은 것을 고르는 게 아니라 지금 이 문제에 맞는 것을 고르는 일이라는 걸 배웠다.

깨달은 점

가장 인상 깊었던 건 RAG를 AWS Knowledge Bases로 구축할 때였다. 처음엔 관리형 서비스가 편하니까 그냥 쓰면 된다고 생각했다. 그런데 막상 써보니 청킹 전략을 조정하거나 page 번호 같은 커스텀 메타데이터를 넣을 수가 없었다. 교과서 RAG에서 "몇 페이지 내용 기반의 답변"이라는 출처 정보는 교육 서비스에서 신뢰도의 핵심이었는데, 관리형 서비스가 그 유연성을 막고 있었다. 결국 직접 파이프라인을 구축했고, 각 chunk에 page 메타데이터를 직접 심었다. "편리함"과 "제어권"은 트레이드오프라는 걸, 직접 부딪혀보고 나서야 이해했다. 두 번째로 배운 건 Tool Calling의 동작 방식이다. AI가 사용자 명령을 판별해서 함수를 실행한다는 개념은 알고 있었지만, 실제로 그 함수가 Django 서버에서 실행되는 게 아니라 함수명과 파라미터를 프론트로 전달하고 클라이언트에 정의된 함수가 실행되는 구조라는 걸 구현하면서 처음 체감했다. 지도 이동이나 인물 호출 같은 UI 상태 변경은 서버가 아닌 클라이언트만이 할 수 있다는, 당연하지만 직접 설계해보지 않으면 모르는 사실이었다.

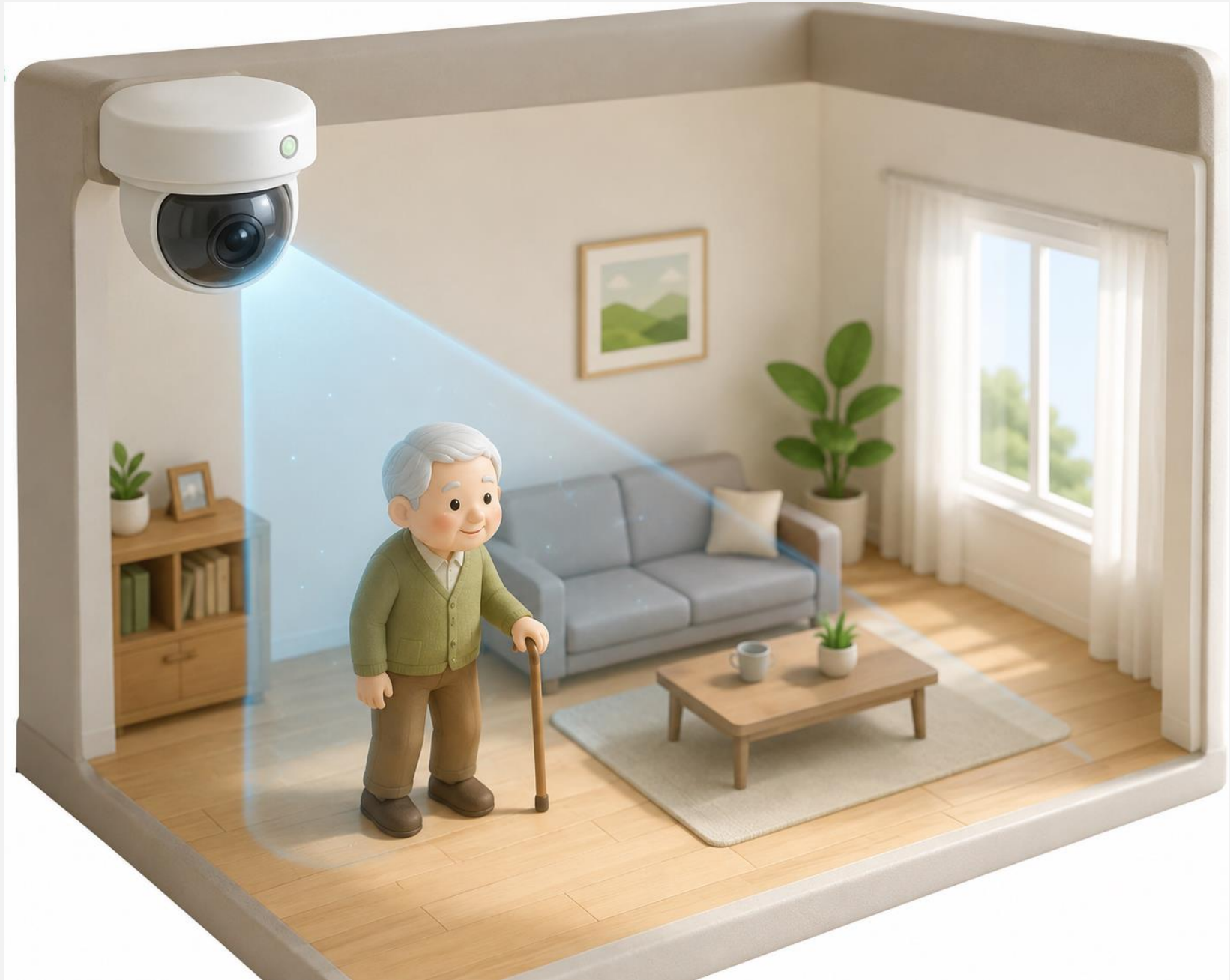
세 번째는 RDS와 Redis를 다르게 취급해야 한다는 점이다. 처음엔 "이중화"를 모든 컴포넌트에 동일하게 적용해야 한다고 생각했다. 하지만 사용자 데이터를 담는 RDS와 세션·캐시를 담는 Redis는 장애 시 복구 방식이 근본적으로 다르다. RDS는 데이터가 사라지면 복구가 불가능하지만, Redis는 장애가 나도 재로그인·재조회로 충분히 복원된다. 모든 걸 똑같이 이중화하는 게 아니라 데이터의 성격과 복구 가능성을 기준으로 판단해야 한다는 걸, 아키텍처를 직접 설계하면서 배웠다.

PROJECT.5

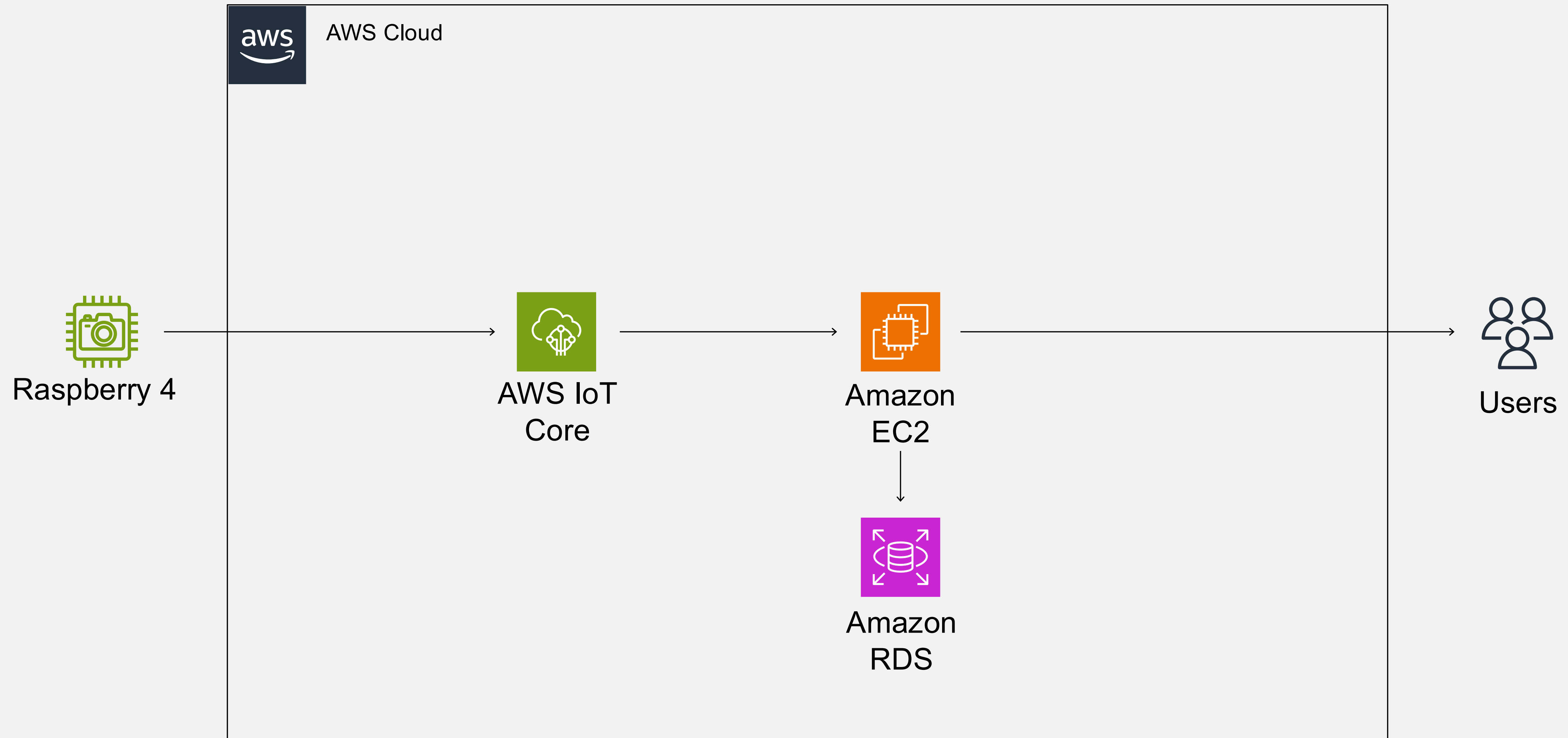
졸업 논문

CNN 기반 노인 낙상 감지 실시간 모니터링 시스템

05



프로젝트 기간	2025.05.01 ~ 2025.11.21
프로젝트 현황	개발 완료
팀 구성	개인 프로젝트
역할	AI 모델 경량화 및 엣지 디바이스 최적화
졸업논문	졸업논문 URL
Git	https://github.com/Hong9915/senior_monitoring_project
실험코드	실험코드 Colab



구분	항목	내용	
Raspberry Pi 4	H/W	CPU	CPU : Broadcom BCM2711
		RAM	RAM : 8GB LPDDR4-3200
	S/W	OS = Raspberry Pi OS (Debian 기반)	
	S/W	Python == 3.9.12 opencv-python ==4.10.0.84 torch == 2.0.1 torchvision == 0.15.2	
Google Coral USB Accelerator	H/W	4TOPS (Google Edge TPU Processor)	
	S/W	Python == 3.9.12 edge-tpu-silva == 1.0.4	
	Power consumption	2W	

Web

HTML JS CSS

django

MariaDB

PERD

Raspberry Pi

OpenCV

ultralytics YOLOv8

AI 기능 구현 및 인프라

데이터셋 구축

Roboflow 공개 낙상 데이터셋 + 커스텀 데이터 추가
총 9,254장 / Falling · Failed · Standing 3클래스 라벨링

YOLOV8N 파인튜닝

Google Colab Pro (Tesla T4 GPU)
100 epoch · batch 32 · 입력 640X640

3단계 경량화 파이프라인

Pruning 10% -> TFLite FP32 변환 -> INT8 Quantization
모델 크기 6..5MB -> 1.8MB (72% 감소)

엣지 디바이스 배포

Raspberry Pi 4 + Google Coral USB Accelerator (Edge TPU)
실시간 추론 11fps 달성

DJANGO 백엔드 구축

Raspberry Pi -> REST API JSON POST 수신 -> DB 저장
heartbeat 헬스체크 + 토큰 기반 인증 설계

모니터링 대시보드

실시간 자세 상태 표시 · 낙상 이력 조회 · 통계분석
Chart.js 활용 / 반응형 웹 디자인

Why — 각 기술 선택의 근거

Q 왜 YOLOv8n 이었나?

YOLOv5n / YOLOv5s / YOLOv8n / YOLOv8s 4개 모델 비교 실험.
YOLOv8n이 파라미터 3.2M, FLOPs 8.7G로 가장 낮은 연산량에서 mAP50 37.3%를 달성 → 엣지 환경 정확도·속도 균형 최적

Q 왜 Pruning 10%만?

10% pruning 시 mAP50이 3.3%p 소폭 감소했으나, mAP50-95는 오히려 4.6% 향상 (regularization 효과).
20% 이상은 mAP50이 33.8%p 급락 → 10%를 최적값으로 선정

Q 왜 INT8 Quantization?

FP32 YOLOv8n은 Raspberry Pi 4 CPU에서 실행 불가.
INT8 양자화 후 메모리 68% 절감 (250MB → 80MB), Coral TPU로 CPU 부하 분산 → 11 FPS 실시간 추론 달성

최적화 전후 비교 및 최종 결과

최적화 단계	모델 크기	mAP50	Raspberry Pi 실행	FPS
YOLOv8n (원본)	6.5 MB	0.96	❌ 불가	-
Pruning 10%	5.9 MB	0.93	❌ 불가	-
TFLite FP32	3.4 MB	0.90	❌ 불가	-
INT8 Quantization + TPU	1.8 MB	0.89	✅ 가능	11 FPS

8h+

연속 안정 동작 | 메모리 누수 없음

4.8W

전력 소비 | USB-C 15W 규격 내

68%

메모리 절감 | 250MB → 80MB

프로젝트 회고

이 프로젝트를 시작할 때 목표는 단순했다. "노인 낙상을 실시간으로 감지하는 시스템을 만들자." 하지만 막상 들어가 보니, 진짜 어려운 건 모델을 잘 학습시키는 게 아니라 **그 모델을 현실의 제약 안에서 돌아가게 만드는 일**이었다.

처음 학습시킨 YOLOv8n은 mAP50 0.96으로 정확도가 충분히 높았다. 그런데 이 모델은 라즈베리파이 4에서 아예 실행조차 되지 않았다. 정확도가 높아도 돌아가지 않으면 의미가 없다는 걸, 코랩의 GPU 환경에서 라즈베리파이로 모델을 옮기는 순간 절감했다. 결국 이 프로젝트의 핵심은 얼마나 정확한가가 아니라 얼마나 정확도를 지키면서 가볍게 만들 수 있는가라는 트레이드오프 싸움이었다.

Pruning, TFLite 변환, INT8 양자화를 단계적으로 적용하면서 모델 크기를 6.5MB에서 1.8MB로, 약 72%까지 줄였다. 정확도는 0.96에서 0.89로 7% 정도 떨어졌지만, 실행 불가능하던 모델이 11 FPS로 실시간 동작하게 된 걸 보면 충분히 합리적인 교환이었다고 생각한다.

깨달은 점

가장 인상 깊었던 건 10% Pruning에서 오히려 mAP50-95가 올라간 현상이었다. 처음엔 데이터를 잘못 본 줄 알았다. 가지치기를 했는데 일부 지표가 좋아진다는 게 직관과 반대였기 때문이다. 찾아보니 적절한 수준의 pruning이 과적합된 불필요한 파라미터를 제거해 일종의 정규화 효과를 낸다는 거였다. "덜어내는 것이 때로는 더 나은 성능을 만든다"는, 모델 최적화에서의 작지만 중요한 교훈을 직접 실험으로 확인한 순간이었다.

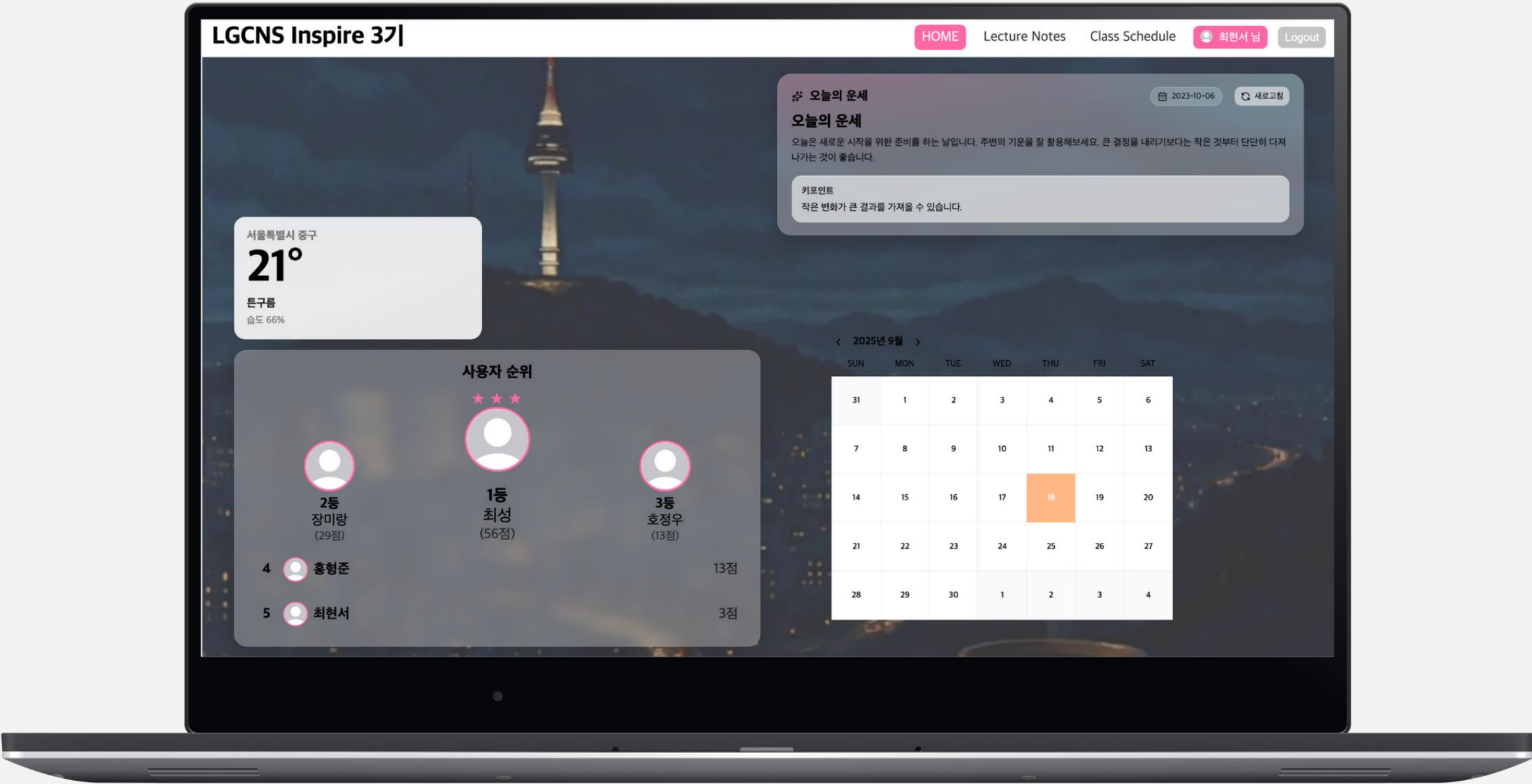
두 번째로 배운 건 **이론적 최적값과 현장의 최적값이 다르다**는 점이다. 20% pruning은 수치상 더 가벼운데, 성능이 임계점을 넘어 급격히 무너졌다. 어디까지가 안전한 경계인지는 결국 직접 여러 비율을 돌려보고 비교해야만 알 수 있었다. 논문 한 줄로 정리되는 "10%가 최적"이라는 결론 뒤에, 사실은 수많은 실패한 실험이 있었다.

세 번째는 **시스템은 모델 하나로 완성되지 않는다**는 것이다. 모델을 경량화하는 데 매달리다 보면 잊기 쉽지만, 결국 보호자가 보는 건 Django 웹 대시보드 화면이다. 엣지 디바이스에서의 추론, 서버와의 통신, 데이터베이스 설계, 웹 시각화까지 전 과정을 직접 엮으면서 "엔드 투 엔드로 동작하는 하나의 제품을 만든다"는 감각을 처음으로 가져볼 수 있었다.

PROJECT.6

Spring Boot 기반 AI 블로그 서비스

06



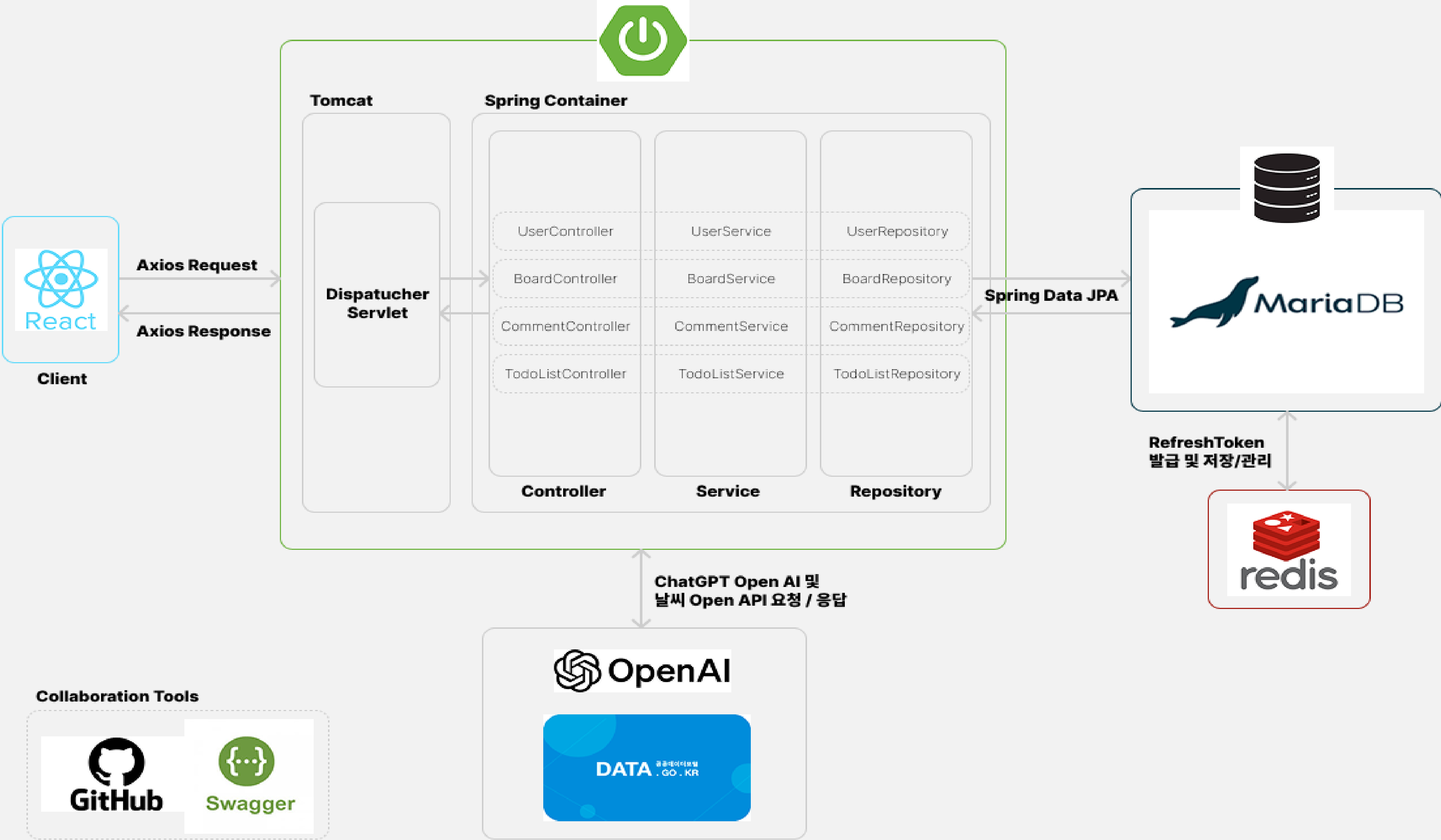
프로젝트 기간2025.09.08 ~ 2025.09.19

프로젝트 현황개발 완료

팀 구성프론트 3명 백엔드 3명

역할팀장 및 백엔드

Git<https://github.com/aaahyunseo/inspire3-team6-pj1>



COMMENT		COMMENT		
	commentId	commentId	Domain	INT
	board	boardId	Domain	BoardEntity
	user	userId	Domain	UserEntity
	content	content	Domain	VARCHAR(100)
	createdAt	createdAt	Domain	DATETIME

USER_RANK		USER_RANK		
	userId	userId	Domain	LONG
	postCount	postCount	Domain	INT
	commentCount	commentCount	Domain	INT
	score	score	Domain	BIGINT
	updateAt	updateAt	Domain	DATETIME

BOARD		BOARD		
	boardId	boardId	Domain	INT
	userId	userId	Domain	LONG
	title	title	Domain	VARCHAR(100)
	content	content	Domain	VARCHAR(500)
	createdAt	createdAt	Domain	DATETIME
	updatedAt	updatedAt	Domain	DATETIME
	viewCount	viewCount	Domain	INT
	likeCount	likeCount	Domain	INT

USER		USER		
	userId	userId	Domain	LONG
	email	email	Domain	VARCHAR(100)
	passwd	passwd	Domain	VARCHAR(100)
	name	name	Domain	VARCHAR(100)
	birthday	birthday	Domain	DATETIME
	createdAt	createdAt	Domain	DATETIME

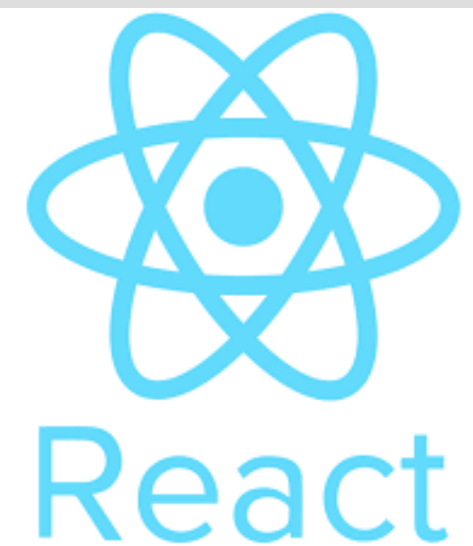
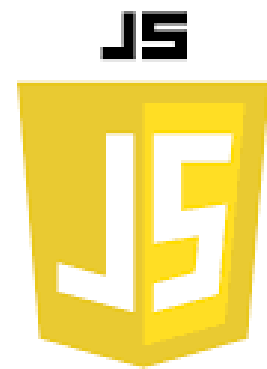
BOARD_LIKE		BOARD_LIKE		
	id	id	Domain	UUID
	board	board	Domain	BoardEntity
	user	user	Domain	UserEntity

BOARD_HASHTAG		BOARD_HASHTAG		
	id	id	Domain	UUID
	boardId	boardId	Domain	INT
	hashtagId	hashtagId	Domain	INT

TODOLIST		TODOLIST		
	todoId	todoId	Domain	LONG
	user	user	Domain	UserEntity
	content	content	Domain	VARCHAR(200)
	done	done	Domain	BOOLEAN
	startDate	startDate	Domain	DATETIME
	endDate	endDate	Domain	DATETIME

HASHTAG		HASHTAG		
	hashtagId	hashtagId	Domain	INT
	name	name	Domain	VARCHAR(100)

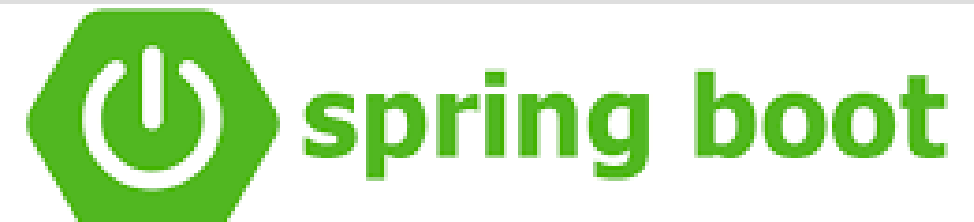
FrontEnd



A X I O S



BackEnd



Collaboration Tools



팀장 | 협업 표준 설계 및 백엔드 개발

회원 인증/권한 관리

Spring Security와 JWT를 활용한 회원 인증 및 권한 관리를 통해 안전한 접근 제어를 제공하고, 비밀번호는 BCrypt로 암호화하여 보안을 강화함.

OPEN AI & OPEN API

OPEN AI 연동 기능을 통해 게시글의 강의 노트를 요약하고 피드백을 제공, 공공데이터 포털 OPEN API를 연동 날씨에 따른 페이지 배경화면 제공

Git Convention

커밋 메시지를 feat, fix, docs, chore, refactor 등 타입 prefix로 시작하도록 통일했다. 커밋 히스토리만 봐도 어떤 작업인지 즉시 파악할 수 있어 코드 리뷰와 디버깅 속도가 높아졌다

Branch Strategy

Git Flow 방식을 채택했다. main(배포), develop(통합), feature/기능명(개발) 3단계로 브랜치를 분리해, 기능 개발 중 main 브랜치가 오염되는 사고를 원천 차단했다.

Code Convention

Java 네이밍 규칙과 들여쓰기, 주석 작성 기준을 통일했다. 서로 다른 스타일로 작성된 코드가 합쳐질 때 생기는 가독성 저하를 방지했다.

API Convention

REST 규칙 기반으로 URL은 복수형 명사(/users), 행위는 HTTP 메서드로 구분하도록 정의했다. Swagger를 연동해 API 명세를 코드와 함께 실시간으로 관리했고, 프론트엔드 팀이 별도 문서 없이 Swagger만으로 연동 작업을 진행할 수 있었다.

프로젝트 회고

이 프로젝트에서 내가 가장 먼저 한 일은 코드를 짜는 게 아니었다. 팀장을 맡으면서 "6명이 2주 동안 충돌 없이 같은 방향을 바라보려면 무엇이 먼저 필요한가"를 생각했고, 개발 시작 전날 Git Convention, Branch Strategy, Code Convention, Naming Convention, API Convention 5가지를 직접 정의해서 팀 전체에 공유했다.

이전 팀 프로젝트 경험에서 규칙 없이 시작했다가 나중에 코드 스타일이 뒤섞이고, 브랜치가 꼬이고, API 스펙이 맞지 않아 프론트-백엔드 사이에서 계속 질문이 오가는 상황을 봤다. 그 비용이 얼마나 큰지 알고 있었기 때문에, 이번엔 처음부터 구조를 잡고 들어가는 게 맞다고 판단했다. Swagger를 연동해 API 명세를 코드와 함께 실시간으로 관리한 것도 같은 이유였다. 프론트엔드 팀이 "이 API 응답 형식이 뭐예요?"라고 물어보는 대신 Swagger를 열어보면 됐고, 그만큼 소통 비용이 줄었다.

깨달은 점

가장 크게 배운 건 좋은 팀장은 가장 먼저 코드를 짜는 사람이 아니라 팀이 막힘 없이 달릴 수 있는 길을 먼저 닦는 사람이라는 것이다. Convention을 정의하는 데 쓴 몇 시간이, 나중에 발생할 수 있는 수십 시간의 혼선을 막았다. 규칙은 제약이 아니라 속도를 내기 위한 도구였다.

두 번째로 배운 건 문서는 소통의 비동기화라는 점이다. Swagger가 없었다면 프론트 팀원이 궁금할 때마다 백엔드에 직접 물어봐야 했다. Swagger가 있으니 내가 자리를 비워도, 잠들어 있어도 팀원이 필요한 정보를 스스로 찾을 수 있었다. 좋은 문서 하나가 사람 한 명을 대신한다는 걸 체감했다.